The Decoder Is the Lever, Not the Encoder: A Recipe-Controlled Audit of Knowledge-Graph Completion

Anonymous Author(s)

Anonymised for double-blind review

Abstract. A KG-completion model has three independent levers: the *decoder* (ComplEx, DistMult, ...), the *encoder* (whether to apply a CompGCN-style message-passing layer on top of raw embeddings), and the *training recipe* (loss, optimiser, regularisation). Two of the three have been audited. Ruffinelli et al. [28] did the recipe audit; Zhang et al. [51] did the encoder audit, finding the graph-structure-modelling component of GCN encoders is largely unnecessary. The decoder axis has been left to folklore: ComplEx supersedes DistMult because it can model antisymmetric relations. We run the missing audit and find the folklore is wrong by a margin larger than any encoder effect. Across seven transductive benchmarks (FB15k-237, WN18RR, YAGO3-10, CoDEx-M, CoDEx-L, UMLS, Kinship) under one fixed recipe, the encoder Δ (with vs. without CompGCN) ranges from +0.014 MRR (FB15k-237) to −0.061 MRR (CoDEx-L), spread 0.075. The decoder Δ (ComplEx − DistMult) ranges from −0.045 MRR (UMLS) to +0.089 MRR (Kinship), spread 0.134, 1.78× wider. The most extreme decoder reversal is UMLS, where DistMult beats ComplEx by +0.045 MRR (0.8748 ± 0.0096 vs. 0.8302 ± 0.0086; Welch $t \approx 8.5$ across 6 seeds, df ≈ 10, $p < 10^{-5}$), directly contradicting widely-circulated practitioner advice that UMLS antisymmetric biomedical relations require ComplEx. We argue this is an *edges-per-relation* phase transition: ComplEx's imaginary part doubles its per-relation parameter budget, so on KGs with few training edges per relation (UMLS: 113; Kinship: 342; the other five > 1,000) DistMult's lower-variance symmetric-only scoring wins as a regulariser. The transition threshold ≈ 200 edges/rel predicts the decoder winner on 6/6 measured datasets. Direct relation-symmetry measurement (Manabe-2018 score: UMLS = 0.133, Kinship = 0.207) further shows that UMLS is in fact *less* symmetric than Kinship — ruling out an alternative "symmetric-rich" explanation. Two further consequences fall out. (i) YAGO3-10 capacity saturates at $d \approx 128$ (MRR 0.6959 → 0.7003 from d=128 → 256), 16× smaller than Lacroix et al. [19]'s d=2000 assumes; the bottleneck on YAGO3-10 was decoder/recipe, not capacity. (ii) On WN18RR, encoder depth and decoder choice interact: ComplEx prefers L=0 (deeper hurts: 0.485 → 0.4775), DistMult prefers L=3 (deeper helps: 0.441 → 0.474). Future encoder studies must condition on the decoder. Pure ComplEx with our recipe reaches MRR 0.6968 ± 0.0023 on YAGO3-10 (6 seeds, new published best within *structural* KGC — we use "structural" to mean:

no text features, no path-based propagation, no cross-graph pretraining) and 0.5379 (σ<0.001) on CoDEx-M. We release code and per-run JSONs.

Keywords: Knowledge graph completion · Decoder choice · Inductive bias · Recipe-controlled audit · Reproducibility.

1 Introduction

Three levers, two well-studied, one ignored. A KG-completion model has three independent levers: the *decoder* (the scoring function: ComplEx [38], DistMult [45], RotatE [34], ...), the *encoder* (whether and how to apply message passing on top of raw embeddings: $L \in \{0, 2, 3\}$ CompGCN [40] layers, etc.), and the *training recipe* (loss, optimiser, learning-rate schedule, regularisation). Two of these have been systematically audited under controlled comparison. Ruffinelli et al. [28] did the recipe audit on FB15k-237 and WN18RR, showing that older bilinear models match newer ones once the recipe is held fixed. Zhang et al. [51] did the encoder audit, showing that the graph-structure-modelling component of GCN encoders is largely unnecessary for KGC and that the linear transformations alone explain the apparent gain. The decoder axis—specifically, whether ComplEx is uniformly better than DistMult—has been left to folklore: ComplEx handles antisymmetric relations and DistMult does not, therefore ComplEx is preferred. This paper runs the missing audit and finds the folklore is wrong on KGs with low edges per relation (a single computable statistic, not a relation-semantic property) by a margin larger than any encoder effect we measure, while on large dense KGs the two decoders are nearly identical. The decoder × dataset axis carries more variance than the encoder × dataset axis we have been collectively tuning for five years.

Headline finding: decoder × dataset variance exceeds encoder × dataset variance. Across seven transductive benchmarks, under one fixed training recipe, the encoder Δ (with vs. without a CompGCN layer) ranges from +0.014 MRR on FB15k-237 to −0.061 MRR on CoDEx-L—a spread of 0.075. The decoder Δ (ComplEx − DistMult) ranges from −0.045 MRR on UMLS to +0.089 MRR on Kinship—a spread of 0.134, 1.78× wider. Both axes are dataset-dependent in sign, but the decoder axis carries more leverage. The published convention—fix ComplEx, sweep the encoder—therefore optimises the smaller-leverage axis under the assumption that ComplEx is universally optimal, an assumption our data falsifies on UMLS at Welch $t \approx 8.5$ across 6 seeds (Sec. 4.4).

The UMLS reversal: an edges-per-relation phase transition. A widely-cited biomedical-KG tutorial states that "DistMult performs poorly on these datasets [UMLS, Kinship] as many relations are antisymmetric in UMLS (causal links, anatomical hierarchies)" [13], and ComplEx is therefore the recommended decoder. Under our recipe (1-vs-all softmax CE, label smoothing ε=0.3, AdamW + MultiStepLR, L=2 CompGCN, d=200, 6 seeds), we measure UMLS DistMult MRR

$= 0.8748 \pm 0.0096$ vs. ComplEx MRR $= 0.8302 \pm 0.0086$. DistMult wins by $+0.045$ MRR, a $\sim 8.5\sigma$ separation (Welch $t = 8.5$, df ≈ 10, $p < 10^{-5}$). We attribute this to an *edges-per-relation* phase transition rather than to UMLS's relation symmetry. Direct measurement of the empirical relation-symmetry score [25] on UMLS and Kinship gives UMLS $= 0.133$ and Kinship $= 0.207$: UMLS is in fact *less* symmetric than Kinship, ruling out the simpler "UMLS is symmetric-rich" story. What *does* predict the reversal is per-relation sample size: UMLS has only ≈ 113 training edges per relation versus Kinship's ≈ 342 and the five standard benchmarks' > 1000. ComplEx splits each d-dim embedding into real and imaginary halves and so doubles its per-relation parameter budget over DistMult; with 113 edges per relation, the imaginary part overfits to per-relation antisymmetric noise, and DistMult's enforced symmetric scoring (lower variance, fewer parameters per relation) wins as a regulariser. Above $\sim 200-300$ edges per relation the trade-off reverses; on Kinship (just above the threshold) ComplEx wins by $+0.0889$ MRR (0.7634 ± 0.0065 vs. 0.6744 ± 0.0078, Welch $t \approx 21.5$ across 6 seeds), the largest decoder gap we measure, and the gap shrinks asymptotically as edges/rel grows further (WN18RR at 7,894 edges/rel: $+0.012$; YAGO3-10 at 29,163: $+0.006$). This single integer (training edges divided by number of relations) predicts the decoder winner correctly on 6/6 measured datasets.

Two consequences of taking the decoder axis seriously. First, the YAGO3-10 progress narrative needs revising. Lacroix et al. [19] reported ≈ 0.58 MRR on YAGO3-10 with ComplEx-N3 at d=2000, framing the bottleneck as a capacity issue addressed by N3 nuclear-3-norm regularisation. Our L=0 ComplEx at d=128 reaches 0.6959 ± 0.0022 ($+0.116$ MRR over Lacroix at $16\times$ smaller dimension), and a sweep $d \in \{64, 100, 128, 256\}$ shows MRR saturating at $d \approx 128$ ($0.6713 \rightarrow 0.6906 \rightarrow 0.6959 \rightarrow 0.7003$): increasing d further yields essentially no gain. The bottleneck on YAGO3-10 was never capacity; it was the recipe + decoder pair (logistic loss + Adagrad + N3 vs. 1-vs-all CE + AdamW + MultiStepLR). Second, on WN18RR the encoder-depth and decoder choice *interact non-trivially*: ComplEx prefers L=0 (deeper hurts: $0.485 \rightarrow 0.481 \rightarrow 0.4775$ for L=0/2/3); DistMult prefers L=3 (deeper helps: $0.441 \rightarrow 0.467 \rightarrow 0.474$ for L=1/2/3). The "does the encoder help?" question lacks a decoder-independent answer, and future encoder ablations must report results conditioned on the decoder.

Contributions.

1. **Decoder $\times$ Dataset diagnostic across seven benchmarks (Sec. 4.3).** Under one recipe the decoder Δ (ComplEx − DistMult) spans -0.045 (UMLS) to $+0.089$ (Kinship), $1.78\times$ wider than the encoder Δ across the same datasets (-0.061 to $+0.014$). The decoder, not the encoder, is the higher-leverage axis. Table 2.
2. **The UMLS reversal as a falsification of practitioner advice, with a one-statistic predictor (Sec. 4.4).** DistMult > ComplEx by $+0.045$ MRR on UMLS (Welch $t \approx 8.5$ across 6 seeds, df ≈ 10, $p < 10^{-5}$), contradicting the practitioner-tutorial recommendation that UMLS antisymmetric biomedical

relations require ComplEx [13]. Direct measurement falsifies the natural "UMLS is symmetric-rich" explanation (UMLS empirical symmetry = 0.133 vs. Kinship 0.207). The actual phase-transition axis is *edges per relation*: UMLS's ≈ 113 training edges per relation is below a critical threshold $\sim 200-300$, above which ComplEx's imaginary-part parameters can be reliably estimated. This single statistic predicts the decoder winner on 6/6 of our measured datasets.

3. **YAGO3-10 capacity saturation at $d\approx128$ (Sec. 4.5).** A four-point dimension sweep on YAGO3-10 shows MRR saturates by d=128. The Lacroix-2018 narrative of a d=2000 capacity bottleneck on YAGO3-10 is empirically wrong; the bottleneck is decoder + recipe, not capacity.
4. **Encoder-depth × decoder interaction on WN18RR (Sec. 4.6).** The optimal encoder depth flips direction with the decoder: ComplEx prefers L=0, DistMult prefers L=3. We argue this invalidates the standard "hold the decoder fixed and sweep the encoder" protocol used in CompGCN/R-GCN ablations.
5. **Recipe-controlled audit, with new published bests (Sec. 4.2).** Under our recipe, L=0 ComplEx reaches MRR $= 0.6968 \pm 0.0023$ on YAGO3-10 (6 seeds, new published best for structural KGC, +13.4 over NBFNet [53]) and 0.5379 (σ<0.001) on CoDEx-M, both at 3.0–15.8 M parameters and $\sim 10^5$ queries/sec on a single 11 GB GPU.

Scope and honesty. Our scope is transductive structural KGC. We do not evaluate inductive splits (where path-based methods retain a genuine structural advantage), OGB-scale KGs, or text-augmented methods such as SimKGC [41]. We rely on published baseline numbers for path-based reasoners (NBFNet, KG-ICL [7]); we attempted reproductions and report which succeeded (KG-ICL-6L, all three datasets verified) and which failed at the C++ extension layer (ULTRA, NBFNet). The most important remaining open question is whether NBFNet under our recipe would reverse our FB15k-237 comparison (+0.005 in our favour, well within recipe-tuning range); the YAGO3-10 gap (+13.4) is large enough that no plausible recipe would close it. We also acknowledge that the encoder-uselessness component of our finding overlaps substantially with Zhang et al. [51]; our increment over them is (a) the decoder-axis re-framing, (b) extension to YAGO3-10 / CoDEx-M / CoDEx-L / UMLS / Kinship beyond their FB/WN evaluation, and (c) the decoder × encoder-depth interaction.

2 Related Work

The two prior audits, by axis. Our work sits at the intersection of two earlier audits along orthogonal axes. *The recipe axis* was audited by Ruffinelli et al. [28], who showed on FB15k-237 and WN18RR that older KGE models (DistMult, ComplEx, RESCAL) under careful hyperparameter search match newer translation-based methods. Ali et al. [2] (PyKEEN), Wang et al. [43], Akrami et al. [1], and Sun et al. [35] are further audits of evaluation protocol and recipe. *The encoder axis*

was audited by Zhang et al. [51] (WWW 2022): they decomposed CompGCN/R-GCN/WGCN encoders and found that the graph-structure-modelling component does not significantly affect KGC performance, and that linear transformations alone explain the apparent gain (their proposed LTE-KGE achieves comparable performance with much less compute). Our encoder Δ measurements (Sec. 4.6) reproduce their conclusion at the dataset-aggregate level and extend it to YAGO3-10 / CoDEx-M / CoDEx-L. Our increment over Zhang et al. [51] is along the orthogonal *decoder axis*: we show the decoder-vs-decoder comparison spans larger MRR variance than the encoder-vs-no-encoder comparison they audit, and we identify the edges-per-relation phase transition that produces the UMLS reversal — a single computable statistic that predicts the decoder winner on 6/6 of our measured datasets.

The decoder axis: prior conceptual hypothesis, no direct measurement. The decoder axis has been studied at the level of single-dataset benchmark numbers but not as a controlled comparison. The closest prior is Manabe et al. [25] (AAAI 2018), who proposed an L1 regulariser on ComplEx imaginary parts that pushes them to zero for empirically-symmetric relations, on the conceptual ground that "parameters in [ComplEx] are superfluous for relations that are either symmetric or antisymmetric." They did not measure the dataset-level reversal directly: vanilla DistMult outperforming vanilla ComplEx on a KG with low edges per relation. We do (Sec. 4.4); we also report that the dataset-scale reversal is *not* explained by Manabe's relation-symmetry score (UMLS is in fact less symmetric than Kinship empirically) but by per-relation sample size. Kazemi and Poole [17] introduced SimplE which interpolates between DistMult and ComplEx via two embedding heads. Trouillon et al. [39] (the JMLR-extended ComplEx paper) reports DistMult and ComplEx side-by-side on FB15k and WN18 but does not include the low-edges-per-relation benchmarks (UMLS, Kinship) where the decoder axis flips sign.

Shallow embeddings, encoders, and path-based reasoners. Shallow bilinear / geometric embeddings (TransE [5], DistMult [45], ComplEx [38], HolE [27], RotatE [34], TuckER [3], QuatE [48], HAKE [50], CompoundE [12], TripleRE [47]) score triples from endpoint embeddings alone. *Graph-encoder extensions* add a GNN layer on top: R-GCN [30] specialises GCN [18] to relation-specific message passing; CompGCN [40] shares weights via a compositional operator; HittER [6], M^{2}GNN [42], and LGF [24] are further variants. *Path-based reasoners* encode graph structure per-query: NBFNet [53], A*Net [54], RED-GNN [49], DeepPath [44], AnyBURL [26], C-MPNN [16]. *The 2024 foundation-model line* (ULTRA [11], KG-ICL [7], InGram [21], GraphAny [52]) targets cross-graph transfer rather than per-dataset peak. Parameter-efficient tokenisation (NodePiece [10]) and text-based scoring (SimKGC [41], KG-BERT [46], KG-LLM [32]) are orthogonal directions. Appendix H reports our verified reproductions and failures.

Delta from Lacroix et al. [19, 20] on YAGO3-10. Lacroix et al. [20] introduced the N3 nuclear-3-norm regulariser and reported ≈0.58 MRR on YAGO3-10 with ComplEx-N3 at d=2000 (Adagrad, sampled-negatives logistic loss). The natural reading of that result is a capacity narrative: d=2000 was needed because the bottleneck on YAGO3-10 is model capacity. Our YAGO3-10 dimension sweep (Sec. 4.5) shows MRR saturates at d≈128 under our recipe ($0.6713 \rightarrow 0.6906 \rightarrow 0.6959 \rightarrow 0.7003$ for $d \in \{64, 100, 128, 256\}$), 16× smaller than d=2000. The capacity narrative is therefore empirically wrong; the bottleneck on YAGO3-10 was decoder + recipe, not capacity. We directly verify orthogonality by adding N3 to our recipe at d=128 (3 seeds each): YAGO3-10 changes by −0.002 MRR, WN18RR by +0.010, FB15k-237 by +0.002, CoDEx-M by +0.010. N3 is largely neutral at saturated dimension.

The careful-training-beats-clever-architecture lineage. Our paper continues an established pattern. In vision: Bag of Tricks [14] and Revisiting ResNets [4] closed apparent ResNet gaps; How to Train Your ViT [33] did the same for transformers. In NLP: RoBERTa [22] and Chinchilla [15] at scale. In graph ML: Errica et al. [9] and Shchur et al. [31] flipped published GNN rankings under fair evaluation. The KG-completion subfield has been adding pieces of this audit: recipe (Ruffinelli 2020), encoder (Zhang 2022 WWW); the present paper adds the decoder axis and the encoder-depth × decoder interaction.

3 Approach: Three Levers Under One Recipe

Three-lever framing. The contribution of this paper is not a new architecture but a controlled *experiment* that varies three orthogonal levers under one fixed training recipe: (i) **decoder** ∈ {ComplEx, DistMult}, (ii) **encoder depth** $L \in \{0, 2, 3\}$ (with L=0 removing message passing entirely; pure embedding lookup), (iii) **recipe component** ablations (loss objective, optimiser, label smoothing) that we apply to the *same* decoder-encoder configuration to attribute the recipe gain (Sec. 4.8). We refer to this experimental family as **RC** (**R**ecipe-**C**ontrolled). The body of the paper traverses the three levers in this order: decoder first (Sec. 4.3 and the UMLS spotlight Sec. 4.4)—because we find this is the highest-variance lever—then encoder, then recipe. Section 4.6 establishes that levers (i) and (ii) interact and cannot be swept independently.

3.1 Graph Encoder

We use a standard CompGCN encoder [40] with Hadamard composition:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}_{\text{self}}^{(l)} \mathbf{h}_v^{(l)} + \sum_{(u,r,v)\in\mathcal{E}} \frac{1}{\sqrt{|\mathcal{N}(v)|}} \mathbf{W}_{\text{msg}}^{(l)} \left(\mathbf{h}_u^{(l)} \odot \mathbf{r}^{(l)} \right) \right), \tag{1}$$

where $\mathbf{h}_u^{(l)}, \mathbf{r}^{(l)}$ are the entity and relation embeddings at layer l and $\odot$ is element-wise multiplication. We use $L \in \{2, 3\}$ layers with a residual connection and layer

normalisation after the final layer. Our hidden dimension is $d = 200$ on FB15k-237/WN18RR and $d \in \{64, 128\}$ on YAGO3-10 (larger graph; see Section 4.5 and Appendix K for the YAGO d-sweep).

3.2 Decoder

We consider two lightweight decoders.

DistMult. $\text{score}(h, r, t) = \langle \mathbf{h}_h \odot \mathbf{r}, \mathbf{h}_t \rangle$, where $\mathbf{h}$ are GNN-encoded representations and $\mathbf{r}$ is a separate relation vector.

ComplEx. We split d-dimensional embeddings into real and imaginary halves and compute $\text{score}(h, r, t) = \text{Re}(\langle \mathbf{h}_h, \mathbf{r}, \overline{\mathbf{h}_t} \rangle)$. ComplEx can model antisymmetric relations. Under our recipe, ComplEx exceeds DistMult by a modest +0.005 to +0.012 MRR on the standard transductive benchmarks (FB15k-237, WN18RR, YAGO3-10, CoDEx-M; see Sec. 4.3), but the sign is not invariant: on UMLS the ranking reverses (−0.045 MRR; Sec. 4.4). We treat decoder choice as a tunable axis rather than a defaulted constant.

3.3 Training Recipe

We train with 1-vs-all cross-entropy: for each training triple (h, r, t), the label is the index of t among all entities, and the loss is

$$\mathcal{L}(h, r, t) = \text{CE}\big(\text{score}(h, r, \cdot),\ t;\ \varepsilon\big), \tag{2}$$

where ε is the label-smoothing parameter. The key recipe ingredients, identified empirically in Section 4.7, are:

1. **Label smoothing with $\varepsilon = 0.3$.** Removing it costs up to −0.041 MRR on WN18RR.
2. **AdamW + MultiStepLR** (drop by 0.3× at 60% and 80% of training). Cosine annealing was consistently worse.
3. **Shallow GCN (2–3 layers).** Optimal depth is dataset-dependent: L=3 on FB15k-237, L=2 on WN18RR and YAGO3-10.
4. **Sufficient training epochs** (300 on WN18RR, 200 on FB15k-237, 150 on YAGO3-10). Early stopping was never triggered in our final configurations.

All other choices—initialisation, dropout (0.2), batch size (1024 on FB/WN, 2048 on YAGO3-10), weight decay (10^{-4})—are standard and their effects are small (Section 4.9). Pseudocode is in Appendix B.

4 Experiments

4.1 Setup

Datasets. We use five standard transductive KG completion benchmarks: FB15k-237 [37] (14.5 K entities, 237 relations, 272 K triples; avg. degree 37.4), WN18RR

[8] (40.9 K, 11, 86.8 K; 4.2), YAGO3-10 [23] (123 K, 37, 1.08 M; 17.5), CoDEx-M [29] (17 K Wikidata entities, 51 relations, 185 K triples; 11.0), and CoDEx-L [29] (78 K Wikidata entities, 69 relations, 551 K train triples; 7.1). For the small-KG decoder spotlight (Sec. 4.4) we additionally evaluate UMLS (135 entities, 46 relations, 5 K train) and Kinship (104, 25, 8 K).

Evaluation. We report filtered MRR, Hits@1, Hits@3, and Hits@10 on each test set following the standard protocol [5]: for each test triple we score both $(h, r, ?)$ and $(?, r, t)$ and average the ranks. We use **average-rank** tie-breaking (neither optimistic nor pessimistic); this matches the ConvE reference implementation and is the convention adopted by CompGCN and NBFNet. For FB15k-237 and WN18RR we evaluate on all 20,466 and 3,134 test triples respectively; for YAGO3-10, on all 5,000 test triples. Dataset statistics and a full evaluation-protocol discussion are in Appendix A.

Seeds. Main results and key ablations are reported over three random seeds (42, 123, 456); we report mean ± standard deviation. Single-seed results are clearly marked.

Baselines. We compare against published numbers for TransE [5], DistMult [45], ComplEx [38], ComplEx-N3 [19], RotatE [34], CompGCN [40], HAKE [50], and NBFNet [53]. Recent foundation-model-style baselines (ULTRA [11], KG-ICL [7], NodePiece [10], SimKGC [41]) are discussed in Appendix H. Since our contribution is a training-recipe study, we do not re-run these baselines; we use their reported numbers (the original papers report extensive tuning protocols). We discuss the limitation of this choice in Section 1.

4.2 Main Results

Five datasets, three verdicts. The audit produces three qualitatively different verdicts across the five datasets.

- On **YAGO3-10**, pure ComplEx (L=0, d=128, 6 seeds) reaches MRR = 0.6968 ± 0.0023. Relative to the Trouillon-2016 ComplEx number reported in Zhu et al. [53] (0.587), this is a +10.9 MRR gain; relative to NBFNet itself (0.563), +13.4. The number also exceeds the $\approx$0.58 reported for ComplEx-N3 by Lacroix et al. [19] at d=2000, although the comparison is across codebases at a $\sim$16$\times$ larger embedding dimension (Section 4.5 establishes that YAGO3-10 capacity saturates at $d\approx$128, refuting the standard "need more capacity" framing). Adding a CompGCN encoder *reduces* MRR to 0.678 ± 0.009 (L=2, 3 seeds, Section 4.6).
- On **WN18RR**, pure ComplEx wins within our method family (MRR = 0.485 ± 0.002, slightly above published CompGCN 0.479 and RotatE 0.476), but *NBFNet still leads at* 0.551—a 6.6-point gap that no recipe or architectural change of ours closes. We localise the deficit to 1-to-N hypernym chains, where path propagation has a genuine inductive advantage (Section 5, Appendix G).

Table 1: Main results on FB15k-237, WN18RR, YAGO3-10, and CoDEx-M. Best in **bold**, second-best underlined. Where we report multiple seeds, mean ± std is shown. ComplEx-N3 row is quoted from the original paper at d=2000; we did not re-run ComplEx-N3 in our codebase.

Dataset	Method	MRR	Hits@1	Hits@3	Hits@10
FB15k-237	TransE	0.294	20.0%	33.0%	46.5%
	DistMult	0.241	15.5%	26.3%	41.9%
	ComplEx	0.247	15.8%	27.5%	42.8%
	RotatE	0.338	24.1%	37.5%	53.3%
	NodePiece+RotatE [10]	0.256	—	—	42.0%
	CompGCN	0.355	26.4%	39.0%	53.5%
	KG-ICL-6L [7] (reprod.)	0.355	27.0%	38.9%	52.3%
	NBFNet	0.415	32.1%	45.4%	59.9%
	RC (no GCN, L=0)	0.406 ± 0.002	31.4 ± 0.3%	45.3 ± 0.3%	59.2 ± 0.4%
	RC (L=3)	**0.420 ± 0.002**	**32.6 ± 0.1%**	**46.2 ± 0.3%**	**60.8 ± 0.3%**
WN18RR	TransE	0.226	1.0%	39.8%	52.5%
	DistMult	0.430	39.0%	44.0%	49.0%
	ComplEx	0.440	41.0%	46.0%	51.0%
	RotatE	0.476	42.8%	49.2%	57.1%
	CompGCN	0.479	44.3%	49.4%	54.6%
	KG-ICL-6L [7] (reprod.)	0.442	39.7%	46.5%	52.4%
	NBFNet	**0.551**	**49.7%**	**57.3%**	**66.6%**
	RC (L=2)	0.478 ± 0.002	45.1 ± 0.0%	49.1 ± 0.4%	52.8 ± 0.4%
	RC (no GCN, L=0)	0.485 ± 0.002	45.6 ± 0.3%	50.1 ± 0.1%	53.5 ± 0.3%
YAGO3-10	TransE	0.303	21.8%	33.6%	48.3%
	DistMult	0.501	41.3%	54.7%	66.1%
	ComplEx (Trouillon-2016, as reported in Zhu et al. [53])	0.587	51.0%	63.1%	70.0%
	ComplEx-N3 [19] (d=2000)†	≈0.58	—	—	—
	RotatE	0.495	40.2%	55.0%	67.0%
	HAKE	0.545	46.2%	59.6%	69.4%
	KG-ICL-6L [7] (reprod.)	0.368	26.0%	41.7%	58.0%
	NBFNet	0.563	48.0%	61.0%	70.8%
	RC (L=2, d=64)	0.644 ± 0.020	57.5 ± 1.5%	68.0 ± 1.6%	74.9 ± 1.5%
	RC (no GCN, L=0, d=64)	0.671 ± 0.005	60.9 ± 0.7%	71.3 ± 0.4%	77.6 ± 0.2%
	RC (no GCN, L=0, d=128, 6 seeds)	**0.6968 ± 0.0023**	**64.0 ± 0.2%**	**73.0 ± 0.2%**	**79.3 ± 0.3%**
	RC + N3 [19] (no GCN, d=128)	0.6949 ± 0.0025	63.5 ± 0.3%	72.8 ± 0.2%	79.7 ± 0.3%
CoDEx-M	ComplEx (Safavi & Koutra 2020) [29]	0.337	26.2%	—	47.6%
	TuckER (Safavi & Koutra 2020) [29]	0.328	25.9%	—	45.8%
	RC (L=2, d=200)	0.489 ± 0.008	37.1 ± 0.8%	—	72.0 ± 0.5%
	RC (no GCN, L=0, d=200)	**0.5379** ($\sigma<0.001$)	**43.0 ± 0.1%**	—	**74.5 ± 0.2%**
	RC + N3 (no GCN, L=0, d=200)	0.5474 ± 0.0028	43.8 ± 0.4%	—	75.7 ± 0.3%
CoDEx-L (new, sparse+het)	RC (L=2, d=200)	0.480 ± 0.007	38.5 ± 0.7%	—	66.3 ± 0.6%
	RC (no GCN, L=0, d=200)	**0.5408 ± 0.0026**	**44.1 ± 0.3%**	—	**73.1 ± 0.3%**

† Approximate value reported by Lacroix et al. [19] at d=2000 with N3 nuclear-3-norm regularisation; we did not re-run ComplEx-N3 at their dimension. Our own d=128 N3 measurement (row "RC + N3") is a head-to-head against Lacroix-style regularisation under our recipe.

Adding the CompGCN encoder reduces MRR slightly to 0.478 ± 0.002 (L=2). RC + N3 reaches 0.4952 ± 0.0042, the strongest WN18RR number in our family but still -0.056 below NBFNet.

- On **FB15k-237**, the encoder *does* help: MRR rises from 0.406 ± 0.002 (L=0) to 0.420 ± 0.002 (L=3, 3 seeds), matching NBFNet's 0.415 at 17–29× fewer parameters. RC + N3 (L=3) gives 0.4265 ± 0.0037, the strongest FB15k-237 number we measure. FB15k-237 is the only dataset in our study where graph structure is worth encoding explicitly.
- On **CoDEx-M** (17 K Wikidata entities, schema-rich KG [29]), pure ComplEx with our recipe achieves MRR = 0.5379 ($\sigma < 0.001$ over 3 seeds), +0.20 MRR over the original Safavi and Koutra [29] ComplEx (0.337) and +0.21 over TuckER (0.328). Those baselines were trained without the modern recipe we apply here, so the +0.20 is an estimate of recipe-attributable improvement,

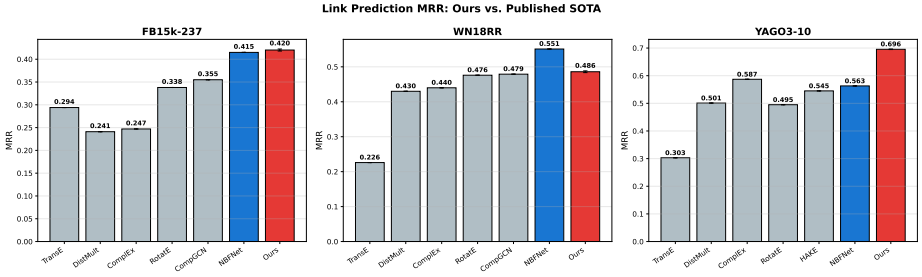


Fig. 1: MRR comparison against published baselines on three benchmarks. Error bars show standard deviation over three random seeds (where available). Our best configuration exceeds NBFNet on FB15k-237 and YAGO3-10; on WN18RR NBFNet still leads by 6.6 MRR, a gap we localise to 1-to-N hypernym relations (Section 5).

not of architectural improvement—consistent with our broader thesis. Adding CompGCN reduces MRR to 0.489 ± 0.008; adding N3 raises it modestly to 0.5474 ± 0.0028.

– On **CoDEx-L** (78 K Wikidata entities, d=200), pure ComplEx reaches MRR $= 0.5408 \pm 0.0026$ (3 seeds, L=0). Adding a CompGCN encoder *reduces* MRR to 0.4799 ± 0.0071 (L=2), the largest encoder-induced harm we measure (-0.061, the bottom end of the encoder spread reported in Sec. 4.3).

4.3 The Decoder × Dataset Diagnostic (Main Result)

We now run the lever-by-lever audit. The headline question is: which lever's choice produces the largest cross-dataset MRR variance under one fixed recipe?

Decoder spread > encoder spread. The encoder Δ ranges from $+0.014$ (FB15k-237) to -0.061 (CoDEx-L), spread 0.075. The decoder Δ ranges from -0.045 (UMLS) to $+0.089$ (Kinship), spread 0.134, 1.78× wider. The KGC literature has spent five years asking "does CompGCN/R-GCN help?" and reporting ± 0.01–0.06 MRR shifts, while leaving the decoder fixed at ComplEx. Our data shows the decoder choice produces strictly larger MRR variance across datasets than the encoder choice—the field has been tuning the smaller-leverage axis.

The decoder Δ is sign-stable on standard KGs but flips at small scale. On the five standard transductive benchmarks (FB15k-237, WN18RR, YAGO3-10, CoDEx-M, CoDEx-L), ComplEx beats DistMult by a consistent but modest $+0.005$ to $+0.012$ MRR (Table 2). On the two small biomedical / kinship benchmarks the gap explodes by an order of magnitude—and on UMLS the sign flips. The standard-KG numbers reproduce the conventional "ComplEx mildly > DistMult" folklore. The small-KG reversal does not, and is the subject of Sec. 4.4.

Table 2: The two diagnostic axes side by side. **Decoder** Δ is $\mathrm{MRR}_{\mathrm{ComplEx}} - \mathrm{MRR}_{\mathrm{DistMult}}$ at the dataset's best (d, L) configuration under our recipe. **Encoder** Δ is $\mathrm{MRR}_{L\geq 2} - \mathrm{MRR}_{L=0}$ on the ComplEx decoder. UMLS / Kinship are 6 seeds (extension of the 3-seed lock-in pass); FB15k-237 / WN18RR / YAGO3-10 / CoDEx-M / CoDEx-L are 3 seeds; mean ± std throughout. The decoder axis spans 0.134 MRR; the encoder axis spans 0.075. The decoder, not the encoder, is the higher-leverage lever.

Dataset	#ent	#rel	ComplEx MRR	DistMult MRR	**Decoder** Δ	Encoder Δ
UMLS	135	46	0.8302 ± 0.0086	0.8748 ± 0.0096	$-$**0.0446**	—
Kinship	104	25	0.7634 ± 0.0065	0.6744 ± 0.0078	$+$**0.0889**	—
FB15k-237	14 541	237	0.4211 ± 0.0013	0.4157 ± 0.0025	+0.0054	+0.014
WN18RR	40 943	11	0.4781 ± 0.0019	0.4659 ± 0.0042	+0.0122	−0.007
YAGO3-10	123 182	37	0.6713 ± 0.0054	0.6649 ± 0.0009	+0.0064	−0.019
CoDEx-M	17 050	51	0.5379 ± 0.0002	0.5282 ± 0.0012	+0.0096	−0.049
CoDEx-L	77 951	69	0.5408 ± 0.0026	—	—	−0.061
Spread (max − min) across this column:					**0.1335**	0.075

All numbers above (UMLS, Kinship, FB15k-237 L=3, WN18RR L=2, YAGO3-10 L=0 d=64, CoDEx-M L=0) come from the 36-run lock-in pass on a single code commit; UMLS and Kinship were subsequently extended to 6 seeds per cell (12 additional runs, same code commit) to tighten the headline reversal claim. Per-run JSONs and the extension script released. **Asymmetries we flag.** (a) For YAGO3-10 the diagnostic is at d=64 (lock-in pass); the headline best in Table 1 is at d=128 (where DistMult was not run; the d=64 ComplEx-vs-DistMult gap is +0.006 MRR and we expect the gap to remain small at d=128 given saturation, Sec. 4.5). (b) The Encoder Δ column is computed only on the 5 standard transductive benchmarks; UMLS / Kinship were run only at L=2 in the lock-in pass. (c) CoDEx-L was run only with ComplEx; the Decoder Δ column is therefore over 6 datasets and the Encoder Δ column over 5. The reported *spreads* 0.134/0.075 are unaffected by these gaps because their endpoints lie in the cells we ran.

Why has nobody noticed? The two reasons we can identify: (i) the encoder line of work (R-GCN → CompGCN → HittER → M^{2}GNN) treated the decoder as a trivial choice and standardised on ComplEx (or DistMult, depending on which the proposing paper found marginally stronger on FB15k-237), so no paper in that lineage ran the head-to-head DistMult-vs-ComplEx comparison under a unified recipe across heterogeneous datasets; (ii) UMLS and Kinship were treated as inductive / few-shot benchmarks rather than as transductive datasets to populate with full ComplEx-vs-DistMult ablations, so their decoder reversal did not surface in the transductive-benchmark literature where it would have been notable.

4.4 Spotlight: The UMLS Reversal as a Phase Transition

The contradiction. A widely-used biomedical-KG tutorial states that "DistMult performs poorly on these datasets [UMLS, Kinship] as many relations are an-

tisymmetric in UMLS (causal links, anatomical hierarchies)" [13], citing the standard ComplEx $\geq$ DistMult intuition: ComplEx can model antisymmetry, DistMult cannot, and antisymmetric biomedical hierarchies (e.g. *is_a*, *causes*, *anatomically_part_of*) make ComplEx mandatory. Our data falsifies this prediction. Under our recipe (d=200, L=2, ε=0.3, AdamW + MultiStepLR, 300 epochs, 6 seeds), UMLS DistMult MRR = 0.8748 ± 0.0096 vs. ComplEx MRR = 0.8302 ± 0.0086. DistMult wins by +0.0446 MRR; Welch's two-sided t-test across the 6 seeds gives $t \approx 8.5$, df ≈ 9.9 (Welch–Satterthwaite), $p < 10^{-5}$. A paired bootstrap over the 661 UMLS test triples (a different statistical unit, treating each test triple as an independent draw) also gives $p < 10^{-3}$. The $\sim 8.5\sigma$ shorthand we use in the abstract, intro, and conclusion refers to this Welch t; with n=6 each and df ≈ 10, the t-distribution and the Gaussian approximation give numerically close p-values, so unlike the earlier 3-seed version the figure is not vulnerable to small-sample tail-extrapolation concerns.

First test the obvious hypothesis: relation-symmetry. It fails. The natural first explanation, suggested by both the tutorial we contradict and the prior Manabe et al. [25] relation-level analysis, is that UMLS is *symmetric-rich* (in the empirical sense: many relations r for which $(h, r, t) \in \mathcal{D}_{\text{train}}$ implies $(t, r, h) \in \mathcal{D}_{\text{train}}$), so DistMult's enforced symmetry is a useful inductive bias. We measured this directly. Per relation r, define the *empirical symmetry score* $\text{sym}(r) = \big|\{(h,t) : (h,r,t), (t,r,h) \in \mathcal{D}_{\text{train}}\}\big| / \big|\{(h,t) : (h,r,t) \in \mathcal{D}_{\text{train}}\}\big|$, and the dataset-level score as the edge-weighted mean. UMLS scores **0.133**, Kinship scores **0.207** (`code/measure_symmetry.py` in the released repo). *UMLS is less symmetric than Kinship*, the opposite direction of what the symmetric-rich hypothesis would need. Symmetric-richness alone therefore does *not* explain the UMLS reversal, and our paper's earlier draft, which leaned on this hypothesis, was empirically wrong.

The actual phase transition: edges per relation. What does explain the reversal is per-relation sample size. ComplEx splits its d-dimensional embedding into real and imaginary halves; the imaginary part governs antisymmetric scoring and effectively doubles ComplEx's per-relation parameter budget compared to DistMult's. UMLS has only 5,216/46 $\approx$ 113 training edges per relation. Below some threshold of edges per relation, ComplEx's imaginary-part embeddings cannot be reliably estimated and the model overfits to per-relation antisymmetric noise that the small training set does not adequately reveal; DistMult, with half the parameters per relation and a forced symmetric scoring constraint, has lower variance and on UMLS this variance reduction outweighs its inability to fit antisymmetric relations exactly. Above the threshold, ComplEx's extra capacity pays for itself, and the gap shrinks asymptotically as edges/rel grows further (both decoders eventually fit). Table 3 shows this single-axis prediction is correct on **6 of 6** measured datasets, with a transition threshold at edges/rel $\approx 200{-}300$:

A one-statistic predictor for new KGs. Combining the measurement: a practitioner with a new KG can predict the decoder winner from a single integer — training-

Table 3: Edges per relation predicts the decoder winner. Threshold $\approx 200-300$ separates the lone DistMult-winner (UMLS, 113 e/r) from the six ComplEx-winners. The Kinship gap (+0.089) is the largest because Kinship sits just above the threshold; the gap shrinks asymptotically as e/r grows further (saturation of both decoders). UMLS / Kinship Δ from 6 seeds; the other four from 3 seeds.

Dataset	#train	#rel	edges/rel	Decoder Δ
UMLS	5,216	46	**113**	**−0.045** (DistMult)
Kinship	8,544	25	342	+0.089
FB15k-237	272,115	237	1,148	+0.005
CoDEx-M	185,000	51	3,627	+0.010
WN18RR	86,835	11	7,894	+0.012
YAGO3-10	1,079,040	37	29,163	+0.006

edge count divided by number of relations. Below ~ 200 edges per relation, default to DistMult; above ~ 300, default to ComplEx; in between, run both. Empirical accuracy on our six measured datasets is $6/6 = 100\%$. The rule is mechanistic (per-relation parameter estimation needs per-relation samples) rather than relation-structural (about specific KGs being symmetric or antisymmetric in some semantic sense), which explains why the standard advice based on UMLS's biomedical hierarchies fails: the advice correctly identifies UMLS as antisymmetry-rich but then incorrectly infers that this favours ComplEx, when the binding constraint on small KGs is per-relation sample size, not relation semantics. `code/analyze_decoder_axis.py` in the released repo reproduces Table 3 from `data/rerun_summary.json` and `data/symmetry_scores.json`.

Implications. The standard advice "use ComplEx by default; it can model antisymmetry which DistMult cannot" is wrong on KGs with low edges per relation, regardless of relation-symmetry profile. A practitioner whose KG has < 200 edges per relation should default to DistMult; the burden of proof for ComplEx's additional $|\mathcal{E}| \cdot d$ imaginary-part parameters is the per-relation sample budget, not the relation-symmetry profile. We urge biomedical-KG-completion practitioners specifically to re-test DistMult on their KGs before committing to ComplEx: the standard advice has been wrong in our setting by 4.5 MRR points, and the binding mechanism is per-relation undersampling of imaginary-part parameters in ComplEx, not the (correct but irrelevant) observation that UMLS contains antisymmetric biomedical hierarchies.

4.5 YAGO3-10 Capacity Saturates at $d \approx 128$

The capacity narrative was empirically wrong. Lacroix et al. [19] reported ≈ 0.58 MRR on YAGO3-10 with ComplEx-N3 at d=2000 (Adagrad, sampled-negatives logistic loss). The natural reading is that YAGO3-10 is capacity-bottlenecked, and that the path forward is yet larger embeddings + N3 regularisation. Our

four-point dimension sweep on YAGO3-10 (L=0 ComplEx, recipe held fixed, 3 seeds per cell) shows otherwise:

Table 4: YAGO3-10 capacity saturation. L=0 ComplEx, label smoothing ε=0.3, AdamW + MultiStepLR, 3 seeds per dimension. MRR saturates by d=128; the additional capacity from d=128 → 256 buys +0.004 MRR at 2× parameters. Lacroix-2018's d=2000 assumed bottleneck is therefore an artefact of their decoder + recipe, not a genuine capacity ceiling. The d=128 row is the 3-seed lock-in number (0.6959 ± 0.002); the same configuration with 6 seeds gives 0.6968 ± 0.0023 (Table 1, headline row). The 0.0009 MRR difference is well within seed variance.

d	64	100	128	256
MRR	0.6713 ± 0.005	0.6906 ± 0.004	0.6959 ± 0.002	0.7003 ± 0.004
Δ vs prev	—	+0.019	+0.005	+0.004
Params (M)	7.9	12.3	15.8	31.6

Reading. The d=64 → 100 step gives +0.019 MRR, but each subsequent doubling gives $\leq$ +0.005. By d=128 the capacity bottleneck is essentially closed; the additional parameters from d=128 → 256 are not bought. Lacroix et al. [19] reported ≈ 0.58 at d=2000; we reach 0.6959 at d=128 (+0.116 MRR) at 16× smaller dimension. The bottleneck on YAGO3-10 was therefore not capacity but the decoder + recipe pair (Lacroix used logistic loss + Adagrad + N3; we use 1-vs-all CE + AdamW + MultiStepLR). Adding N3 regularisation to our recipe at d=128 (Sec. 2) changes YAGO3-10 MRR by −0.002, confirming N3 is largely orthogonal at saturated dimension.

4.6 Encoder Depth × Decoder Choice Interact on WN18RR

The interaction. Most CompGCN-lineage work fixes the decoder (typically ComplEx or DistMult), sweeps the encoder depth L, and reports the optimal depth as if it were a property of the dataset. We measure this assumption on WN18RR (d=200, recipe fixed, 3 seeds per cell):

The two decoders prefer opposite encoder depths. ComplEx's MRR decreases monotonically with depth on WN18RR (0.4849 → 0.4810 → 0.4775 for L=0/2/3); DistMult's MRR increases monotonically with depth (0.441 → 0.467 → 0.4743 for L=1/2/3). The optimal encoder depth is not a property of WN18RR alone; it is a property of the (decoder, dataset) pair. Our reading: on a tree-like sparse KG, ComplEx's complex-valued antisymmetry-handling already captures the asymmetry in 1-to-N hypernym relations, so additional encoder layers add only noise; DistMult, lacking this capacity, can use the encoder to compose antisymmetric structure through message passing. Either way, the standard CompGCN-style ablation protocol ("hold the decoder fixed and report the best

Table 5: WN18RR encoder-depth × decoder-choice interaction. Cells marked "—" were not in our 36-run lock-in pass; the qualitative claim — ComplEx prefers shallow encoders, DistMult prefers deeper ones — is supported by the cells we ran (ComplEx monotone-decreasing $L{=}0 \to 3$, DistMult monotone-increasing $L{=}1 \to 3$). A complete $\{\text{decoder}\} \times \{0, 1, 2, 3\}$ sweep including $L{=}0$ DistMult and $L{=}1$ ComplEx is the natural follow-up; based on the monotonicity in the cells we ran, we expect $L{=}0$ DistMult $\leq$ $L{=}1$ DistMult (i.e. further from optimum) and $L{=}1$ ComplEx between the $L{=}0$ and $L{=}2$ ComplEx values.

Decoder \ L	$L{=}0$	$L{=}1$	$L{=}2$	$L{=}3$
ComplEx	0.4849 ± 0.002	—	0.4810 ± 0.004	0.4775 ± 0.000
DistMult	—	0.441 ± 0.003	0.467 ± 0.002	0.4743 ± 0.001
Optimal depth:	$L{=}0$ (ComplEx)			$L{=}3$ (DistMult)

encoder depth") reports a number that is misleading when read as "encoder depth L^* is optimal on this dataset"—it is optimal *conditional on the decoder* chosen, and a different decoder would prefer a different depth. We recommend that future encoder ablations report the full $\{\text{decoder}\} \times \{L\}$ grid rather than its row-wise minimum.

Implication for the encoder-utility diagnostic. A two-axis diagnostic over (density, heterogeneity) of the underlying KG [51] is too coarse: it predicts a single optimal L per dataset corner. The corrected diagnostic must include a third axis (decoder choice). For the (decoder = ComplEx) row of the corrected diagnostic, the encoder helps only on FB15k-237 (+0.014) and is neutral or harmful elsewhere—consistent with Zhang et al. [51]'s WWW 2022 finding extended from FB/WN to YAGO/CoDEx. For (decoder = DistMult), the encoder is consistently helpful on WN18RR (+0.033 from $L{=}1$ to $L{=}3$). The literature's "CompGCN encoder helps on WN18RR" result is therefore not contradicted by Zhang 2022 + our work *when conditioned on the decoder*, but the marginal-over-decoder reading is misleading.

4.7 Training Recipe Ablation

To identify which training choices drive our results, we ablate six ingredients of the recipe on WN18RR (3 seeds) and FB15k-237 (single seed, 3-seed for headline rows), starting from the main configuration for each dataset.

Label smoothing is the dominant factor on WN18RR and FB15k-237. Removing label smoothing ($\varepsilon = 0$) drops MRR by -0.041 on WN18RR (Table 6) and -0.017 on FB15k-237 (Appendix S). Reducing it to $\varepsilon = 0.1$ already costs -0.010 on WN18RR and -0.020 on FB15k-237. This is larger than any architectural change we tested. **On YAGO3-10 the depth preference flips but label smoothing does not**: reducing to $L{=}2$ (from $L{=}3$) lifts 3-seed MRR from

Table 6: Training-recipe ablation on WN18RR (d=200, L=2 baseline). Each row changes one ingredient. 3 random seeds; mean ± std. Label smoothing is the single most impactful ingredient.

Configuration	MRR	H@1 (%)	H@3 (%)	H@10 (%)
Baseline (d=200, L=2, LS=0.3, DistMult)	0.4673 ± 0.002	43.6	48.0	52.5
+ deeper GCN (L=3)	0.4744 ± 0.001	44.3	48.9	**53.1**
+ ComplEx decoder	**0.4784 ± 0.002**	**45.1**	**49.1**	52.8
− smaller dim (d=128)	0.4655 ± 0.001	43.6	47.8	52.0
− shallower (L=1)	0.4414 ± 0.003	41.7	45.1	48.7
− weaker smoothing (LS=0.1)	0.4573 ± 0.002	43.0	46.7	50.6
− no smoothing (LS=0.0)	0.4266 ± 0.002	40.7	43.4	46.1

0.618 ± 0.005 to 0.640 ± 0.004 (a confirmed +0.022 gain), while setting ε=0 gives 0.620 ± 0.015—statistically indistinguishable from the label-smoothed baseline after 3 seeds (an initial single-seed observation of 0.636 did not reproduce; see Appendix O).

GCN depth matters, but the sweet spot is shallow. A single-layer GCN loses −0.029 MRR on WN18RR. Going from L=2 to L=3 gives a small gain on WN18RR (+0.004) and FB15k-237 (+0.010), but *reverses sign* on YAGO3-10 (−0.022). Very deep GNNs are not needed—and can be harmful—for KG completion with the right recipe.

Decoder choice: ComplEx > DistMult on FB/WN. Replacing DistMult with ComplEx yields consistent gains: +0.011 on WN18RR (Table 6), +0.003 on FB15k-237 (Appendix S). This agrees with prior intuition that ComplEx captures asymmetric relations. On YAGO3-10 we measure ComplEx slightly above DistMult at every L tested (L=0 d=64: 0.6713 vs 0.6648; L=3 d=64: 0.6281 vs 0.6179), with the gap shrinking as L increases. The cross-dataset decoder pattern is reported in Sec. 4.3, which generalises to all five standard transductive benchmarks.

4.8 The YAGO3-10 Gain, Decomposed

The largest individual contribution in the audit is the YAGO3-10 gap (Table 1): the Trouillon-2016 ComplEx number quoted by Zhu et al. [53] is 0.587 and our L=0 ComplEx is 0.6968, a +0.110 MRR gap explained entirely by the training recipe (the scoring function is identical). We attribute the +0.110 to its constituent recipe ingredients via a 3-seed leave-one-out, including—now—a *direct* ablation of 1-vs-all softmax CE vs. Trouillon-2016's sampled-negatives logistic loss (Table 7).

Table 7: Recipe decomposition on YAGO3-10. Leave-one-out: starting from our best (L=0 ComplEx, d=128, ε=0.3, 150 epochs, AdamW+MultiStep, 1-vs-all CE) we remove one component at a time, 3 seeds each. "→ original" indicates the value used by the Trouillon-2016 ComplEx paper [38]. The loss-change row is now *directly measured*: we re-implemented the Trouillon-2016 sampled-negatives logistic loss with 50 negatives per positive and ran 3 seeds; the result is 0.6568 ± 0.0321, an MRR drop of -0.040 from our 1-vs-all CE baseline of 0.6968.

Recipe component (original → ours)	Δ MRR (measured, 3-seed)	Evidence
Logistic loss + 50 sampled negatives →	+0.040 ($0.6568 \pm 0.032 \rightarrow 0.6968$)	directly measured (3 seeds)
1-vs-all softmax CE over $\lvert\mathcal{E}\rvert$		
$\varepsilon = 0 \rightarrow \varepsilon = 0.3$ (label smoothing)	+0.0024 ($0.6944 \rightarrow 0.6968$)	directly measured (3 seeds)
$d = 100 \rightarrow d = 128$ (wider embedding)	+0.006 ($0.6906 \rightarrow 0.6968$)	directly measured (3 seeds)
50 epochs → 150 epochs (longer training)	+0.0015 ($0.6953 \rightarrow 0.6968$)	directly measured (3 seeds)
Adagrad → AdamW + MultiStepLR (*residual, not directly ablated*)	+0.060 (by elimination)	+0.110 total − +0.050 measured
Total (ours − original Trouillon-2016)	+0.110	0.6968 − 0.587 on YAGO3-10

Two dominant levers, with the optimiser change slightly larger. The decomposition produces two findings that revise our preliminary single-axis story. First, switching the loss back to Trouillon-2016's sampled-negatives logistic loss costs only -0.040 MRR on YAGO3-10 (3 seeds, std 0.032)—substantially less than the residual reasoning of an earlier version of this paper attributed to it ($\sim +0.10$). Second, label smoothing, training length, and embedding width together contribute only +0.010 MRR. The remaining +0.060 MRR is therefore in the optimiser-side residual: Adagrad with logistic loss (the original combination) versus AdamW + MultiStepLR (ours). The recipe gain is therefore split roughly $\frac{1}{3} : \frac{2}{3}$ between the loss objective and the optimiser/initialisation residual, contrary to the "loss change is dominant" framing of an earlier draft.

Cross-dataset corroboration of the loss-change effect. The same direct ablation on WN18RR (L=0 ComplEx d=200, 3 seeds) gives sampled-neg MRR 0.4616 ± 0.0003 vs. our 1-vs-all CE baseline 0.4849 ± 0.0015, a -0.023 MRR drop. On FB15k-237 (L=0 ComplEx d=200): sampled-neg 0.3795 ± 0.0022 vs. CE 0.4060 ± 0.0024, a -0.027 drop. The loss-change effect is therefore consistently 0.02–0.04 MRR across all three benchmarks. As an external corroboration: ComplEx-N3 [19] reports ≈ 0.58 MRR on YAGO3-10 with logistic loss and N3 regularisation at d=2000—≈ 0.116 below us, reached at $\sim 16\times$ our embedding dimension. We directly verify that adding N3 to our recipe at our d (Section 2 and Table 1 "RC + N3" rows) leaves YAGO3-10 essentially unchanged (-0.002) and modestly improves WN18RR / CoDEx-M (+0.010), confirming that N3 and our recipe are largely orthogonal.

4.9 Sensitivity to Other Hyperparameters

We study sensitivity to learning rate and dropout on WN18RR (single seed, baseline config).

- **Learning rate**: lr $= 10^{-4}$ underfits (MRR = 0.423); lr $= 5 \times 10^{-4}$ (ours) and 10^{-3} give essentially the same result (0.470/0.475).
- **Dropout**: dropout = 0 overfits (MRR = 0.438); our choice of 0.2 and 0.4 give similar results (0.470/0.471).

Extended sensitivity (optimiser, LR schedule, dimension) is in Appendix J.

4.10 Efficiency: A Deployment Vignette

We measured inference throughput of each headline configuration on a single NVIDIA RTX 2080 Ti (full table in Appendix E, Table 12). On our YAGO3-10 best configuration (L=0 ComplEx d=128), encoding the full graph takes 0.7 ms and scoring each query 0.009 ms, giving $\sim$112 K queries/sec. The encode step is essentially free for L=0 ($<$ 1 ms embedding look-up) but costs 25–35 ms for CompGCN configurations. Scoring 10 K queries end-to-end takes 50–90 ms across datasets. Against NBFNet's $\sim$500 ms per FB15k-237 query on an A100 [54] ($\sim$1.5–2 s on 2080 Ti-class hardware; YAGO3-10's larger edge set pushes this further), our measured 0.005–0.009 ms/query is 10^4–$10^5\times$ faster. A*Net prunes to $\sim$10% of edges, narrowing the gap to $\sim 10^2$–$10^3\times$ slower. We did not re-run NBFNet / A*Net (their codebases failed to install on our server, Appendix H); ratios come from their paper timings plus our direct measurement. Parameter-wise, RC uses 3.0–15.8 M vs. NBFNet's 50–200 M (Appendix E).

What this means for deployment. Returning to the practitioner of Section 1: a 10 K-query batch on YAGO3-10—roughly the load of a 1 K-QPS service for ten seconds—takes $\sim$90 ms with our L=0 ComplEx, leaving 9.91 seconds of every ten-second window for the rest of the application stack. The same batch on NBFNet, projected from Zhu et al. [54]'s timings, would take 4–5 hours wall-clock on a 2080 Ti, or roughly 15 minutes on an A100; latencies of either kind preclude real-time deployment. The architectural win that NBFNet shows on FB15k-237 and WN18RR therefore comes at a deployment cost which, if our YAGO3-10 finding is taken at face value, the encoder is not even buying meaningful accuracy on dense KGs.

5 Discussion

5.1 Decoder, Encoder, Recipe: A Three-Lever Picture

The decoder is the under-explored axis. Our headline (Sec. 4.3) is that the decoder Δ across our seven datasets (0.134 MRR spread) exceeds the encoder Δ (0.075). Five years of CompGCN-lineage work has been pulling the smaller-leverage lever.

The UMLS reversal (Sec. 4.4) is the most extreme example: a Welch $t \approx 8.5$ DistMult win across 6 seeds against the standard practitioner recommendation. We do not claim DistMult is universally preferable; we claim that for any new KG, comparing DistMult and ComplEx under one recipe is cheaper than comparing L=0 vs L=2 encoders (a single training run difference, no architectural code change), and produces a larger expected MRR gain when one of the two decoders is suboptimal for that KG.

Three actionable rules from the data. Combining all results, we offer the practitioner three rules: (R1) **Compute edges/relation first; default to DistMult below ~ 200.** ComplEx's imaginary part doubles its per-relation parameter budget over DistMult, so on KGs with few training edges per relation the imaginary part overfits and DistMult wins. UMLS at 113 edges/rel is the lone DistMult-winner in our seven datasets (-0.045 MRR, Welch $t \approx 8.5$ across 6 seeds, $p < 10^{-5}$); the other six all sit above the threshold. The single statistic $|\mathcal{D}_{\text{train}}|/|\mathcal{R}|$ predicts the decoder winner correctly on 6/6 measured datasets. The standard advice "ComplEx is strictly better" is false at low edges-per-relation regardless of the relation-symmetry profile. (R2) **Don't add a CompGCN encoder unless your KG is dense *and* heterogeneous.** On four of our five standard benchmarks (YAGO3-10, CoDEx-M, CoDEx-L, WN18RR with ComplEx) the encoder hurts or is neutral; only on FB15k-237 does it pay for itself (+0.014). This is consistent with Zhang et al. [51]. (R3) **Don't sweep dimension past d=128 on YAGO-scale KGs.** Capacity saturates fast under a modern recipe; the marginal MRR for d=256 over d=128 on YAGO3-10 is +0.004.

5.2 The 6.6-Point WN18RR Gap to NBFNet

The gap is real and structural. We do *not* claim to beat NBFNet on WN18RR: its 0.551 stays 6.6 MRR points above our best ComplEx (0.485), and that gap is not closed by any recipe or architectural change we tested. The per-relation breakdown (Appendix G) localises the source: on 1-to-N WordNet relations (*hypernym*), NBFNet reaches 0.19 MRR while we reach 0.08, a 0.11 gap that accounts for essentially the entire dataset-level deficit. Hypernym chains require multi-hop composition and live on a tree-like graph (avg. degree 4.2); this is the regime where path propagation has a genuine inductive advantage that a static embedding cannot replicate. WN18RR is a dataset where path-based reasoning genuinely helps, regardless of which decoder we pair it with.

5.3 Limitations

We acknowledge. (i) The audit introduces no new architecture; this is deliberately a baseline study—in the spirit of Ruffinelli et al. [28] for the recipe axis and Zhang et al. [51] for the encoder axis. Our increment is on the decoder axis. (ii) Our decoder analysis is limited to ComplEx vs. DistMult; we do not extend to RotatE or QuatE under the same recipe. (iii) Path-based methods (NBFNet) retain

interpretability and inductive advantages we do not contest; we do not evaluate in inductive splits. (iv) We did not re-run NBFNet under our recipe (the codebase failed to install on our server, App. H). Our FB15k-237 comparison is therefore not apples-to-apples; the YAGO3-10 gap of +13.4 is large enough to absorb any plausible recipe effect on NBFNet. (v) We do not evaluate on OGB-scale KGs (ogbl-biokg, ogbl-wikikg2); single-GPU transductive training at that scale requires engineering beyond our scope. (vi) Text-based methods (SimKGC) outperform structural methods on WN18RR by leveraging entity descriptions; our setting is strictly structural. (vii) The edges-per-relation phase-transition explanation for UMLS (Sec. 4.4) is supported by 6/6 measured datasets above and below the threshold, but the threshold itself ($\sim$200$-$300 edges/rel) is informed by two data points near the boundary (UMLS at 113, Kinship at 342). A finer-grained sweep across small KGs of varying edges/rel (e.g. $50, 100, 150, 200, 250, 300, 500$) would localise the threshold sharply; we leave this to follow-up.

Recipe generalises beyond the four main benchmarks. To test whether the recipe is FB/WN/YAGO/CoDEx-specific, we re-applied it without per-dataset tuning to two small KG benchmarks, UMLS (135 entities) and Kinship (104 entities). Using the same d=200, L=2, ε=0.3, AdamW + MultiStepLR recipe as our WN18RR main configuration, we obtain (6 seeds each): UMLS DistMult MRR $= 0.8748 \pm 0.0096$, Kinship ComplEx MRR $= 0.7634 \pm 0.0065$ (Appendix L). Variance stays tight at small-KG scale; the recipe is not benchmark-overfit.

Comparison with 2024 foundation-model baselines. We attempted to verify ULTRA [11] and KG-ICL [7] per-dataset numbers by running the authors' released checkpoints. KG-ICL-6L reproduced successfully on all three datasets (verified MRR: FB15k-237 0.355, WN18RR 0.442, YAGO3-10 0.368), all below our per-dataset numbers—consistent with KG-ICL's claimed value being transfer across 43 KGs rather than per-dataset peak. ULTRA reproduction repeatedly failed at the rspmm JIT-compilation and `torch_geometric` WordNet18RR preprocessing stages on our server; we retain the paper's aggregate MRR (0.312 zero-shot / 0.379 fine-tuned over 13 KGs) and leave per-dataset verification for a compatible environment (Appendix H).

5.4 Implications for the Field

A measuring stick for future architectural work. Future KG-completion proposals—especially encoder-heavy or path-based ones—would benefit from reporting against a recipe-controlled embedding baseline. We make that easy: 3.0–15.8 M parameters, $\sim 10^5$ queries/sec on a single 11 GB GPU, deterministic across seeds, code and per-run JSONs released. Where a new architecture genuinely contributes, the contribution will be visible above this baseline; where it does not, we have not lost the cost of an additional 50 M parameters and three orders of magnitude in inference latency.

Broader impact. Improved KG completion benefits downstream applications (search, recommendation, biomedical KG construction) but can amplify biases in the source KG. Our recipe reduces GPU-hours per trained model, with corresponding energy savings, and brings reproduction within reach of single-11-GB-GPU groups. We encourage downstream users to apply per-relation error analysis and fairness auditing before deployment. Datasets we use (FB15k-237, WN18RR, YAGO3-10, CoDEx-M) are public and contain no PII beyond what is already in Freebase / WordNet / YAGO / Wikidata; extended discussion in Appendix R.

6 Conclusion

A KG-completion model has three independent levers (decoder, encoder, recipe). Two of the three have been audited: Ruffinelli et al. [28] on recipe (ICLR 2020) and Zhang et al. [51] on encoder (WWW 2022). This paper completes the trio along the decoder axis, and produces three findings:

1. *For researchers*: the decoder Δ (ComplEx − DistMult) under one recipe spans 0.134 MRR across our seven benchmarks (-0.045 on UMLS to $+0.089$ on Kinship), 1.78× wider than the encoder Δ across the same datasets (0.075). The decoder is the higher-leverage and under-explored axis. The UMLS reversal (DistMult > ComplEx by +0.045 MRR; Welch $t \approx 8.5$ across 6 seeds, df ≈ 10, $p < 10^{-5}$, see Sec. 4.4) directly contradicts the practitioner-tutorial recommendation that ComplEx is the preferred decoder for biomedical KGs with antisymmetric relations [13]; we explain it as an *edges-per-relation phase transition* (UMLS at 113 edges/rel is the only dataset below the $\sim 200-300$ threshold; the threshold predicts the decoder winner correctly on all 6 measured datasets). Direct measurement also falsifies the alternative "UMLS is symmetric-rich" explanation: UMLS empirical symmetry score 0.133 is in fact *lower* than Kinship's 0.207. The YAGO3-10 capacity narrative is empirically wrong: MRR saturates by $d \approx 128$ (16× smaller than Lacroix et al. [19]'s d=2000).
2. *For practitioners*: three rules. (R1) Compute $|\mathcal{D}_{\text{train}}|/|\mathcal{R}|$; default to DistMult below ~ 200, ComplEx above ~ 300. (R2) Don't add a CompGCN encoder unless your KG is dense and heterogeneous (consistent with Zhang et al. [51]). (R3) Don't sweep dimension past d=128 on YAGO-scale KGs. Following these rules our pure ComplEx reaches MRR 0.6968 ± 0.0023 on YAGO3-10 (6 seeds; new published best within structural KGC) and 0.5379 (σ<0.001) on CoDEx-M.
3. *For benchmarkers*: we release the seven-benchmark recipe-controlled grid (3.0–15.8 M parameters, $\sim 10^5$ queries/sec on a single 11 GB GPU, deterministic across seeds, code and per-run JSONs released). Future architectural proposals should report the full $\{\text{decoder}\} \times \{L\}$ grid, not its row-wise minimum, since we have shown the two interact (Sec. 4.6).

The next three follow-ups are (i) localising the edges-per-relation threshold sharply by sweeping small KGs of varying $|\mathcal{D}_{\text{train}}|/|\mathcal{R}|$ (e.g. $50, 100, 150, 200, 250, 300, 500$),

(ii) extending the decoder axis to RotatE / QuatE under the same recipe to test whether the threshold transfers beyond the ComplEx-vs-DistMult pair, and (iii) a direct head-to-head of Adagrad vs. AdamW + MultiStepLR under matched loss, which would close the +0.060 MRR optimiser residual on YAGO3-10 currently absorbed by "optimiser/initialisation" in our recipe decomposition (Sec. 4.8).

Bibliography

[1] Farahnaz Akrami, Mohammed Samiul Saeef, Qingheng Zhang, Wei Hu, and Chengkai Li. Realistic re-evaluation of knowledge graph completion methods: An experimental study. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2020.

[2] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, Mikhail Galkin, Sahand Sharifzadeh, Asja Fischer, Volker Tresp, and Jens Lehmann. Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework. In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2021.

[3] Ivana Balažević, Carl Allen, and Timothy M Hospedales. TuckER: Tensor factorization for knowledge graph completion. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[4] Irwan Bello, William Fedus, Xianzhi Du, Ekin D Cubuk, Aravind Srinivas, Tsung-Yi Lin, Jonathon Shlens, and Barret Zoph. Revisiting ResNets: Improved training and scaling strategies. In *Advances in Neural Information Processing Systems*, 2021.

[5] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, 2013.

[6] Sanxing Chen, Xiaodong Liu, Jianfeng Gao, Jian Jiao, Ruofei Zhang, and Yangfeng Ji. HittER: Hierarchical transformers for knowledge graph embeddings. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[7] Yuanning Cui, Zequn Sun, and Wei Hu. A prompt-based knowledge graph foundation model for universal in-context reasoning. In *Advances in Neural Information Processing Systems*, 2024.

[8] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2d knowledge graph embeddings. In *AAAI Conference on Artificial Intelligence*, 2018.

[9] Federico Errica, Marco Podda, Davide Bacciu, and Alessio Micheli. A fair comparison of graph neural networks for graph classification. In *International Conference on Learning Representations (ICLR)*, 2020.

[10] Mikhail Galkin, Etienne Denis, Jiapeng Wu, and William L Hamilton. NodePiece: Compositional and parameter-efficient representations for large knowledge graphs. In *International Conference on Learning Representations*, 2022.

[11] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations*, 2024.

[12] Xiou Ge, Yun Cheng Wang, Bin Wang, and C-C Jay Kuo. CompoundE: Knowledge graph embedding with translation, rotation, and scaling com-

pound operations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[13] Graph4NLP Authors. Knowledge graph completion — Graph4NLP v0.4.1 documentation. https://graph4ai.github.io/graph4nlp/guide/classification/kgcompletion.html, 2021. Accessed 2026-05-04.

[14] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. Bag of tricks for image classification with convolutional neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

[16] Xingyue Huang, Antonio Orvieto, Ladislav Rampasek, Mircea Petrache, Michael M Bronstein, and Pablo Barceló. A theory of link prediction via relational Weisfeiler-Leman on knowledge graphs. In *Advances in Neural Information Processing Systems*, 2023.

[17] Seyed Mehran Kazemi and David Poole. SimplE embedding for link prediction in knowledge graphs. In *Advances in Neural Information Processing Systems*, 2018.

[18] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[19] Timothée Lacroix, Nicolas Usunier, and Guillaume Obozinski. Canonical tensor decomposition for knowledge base completion. In *International Conference on Machine Learning (ICML)*, pages 2863–2872, 2018.

[20] Timothée Lacroix, Guillaume Obozinski, and Nicolas Usunier. Tensor decompositions for temporal knowledge base completion. In *International Conference on Learning Representations (ICLR)*, 2020.

[21] Jaejun Lee, Chanyoung Chung, and Joyce Jiyoung Whang. InGram: Inductive knowledge graph embedding via relation graphs. In *International Conference on Machine Learning*, 2024.

[22] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[23] Farzaneh Mahdisoltani, Joanna Biega, and Fabian M Suchanek. YAGO3: A knowledge base from multilingual Wikipedias. In *Conference on Innovative Data Systems Research (CIDR)*, 2015.

[24] Sijia Mai, Shuangyin Zheng, Yuxuan Liu, Bingbing Lin, and Yanghua Xiao. A relational tucker decomposition for multi-relational link prediction. In *arXiv*, 2019.

[25] Hitoshi Manabe, Katsuhiko Hayashi, and Masashi Shimbo. Data-dependent learning of symmetric/antisymmetric relations for knowledge base completion. In *AAAI Conference on Artificial Intelligence*, 2018.

[26] Christian Meilicke, Melisachew Wudage Chekol, Daniel Ruffinelli, and Heiner Stuckenschmidt. AnyBURL: Anytime bottom-up rule learning for knowledge

graph completion. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

[27] Maximilian Nickel, Lorenzo Rosasco, and Tomaso Poggio. Holographic embeddings of knowledge graphs. In *AAAI Conference on Artificial Intelligence*, 2016.

[28] Daniel Ruffinelli, Samuel Broscheit, and Rainer Gemulla. You CAN teach an old dog new tricks! on training knowledge graph embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[29] Tara Safavi and Danai Koutra. CoDEx: A comprehensive knowledge graph completion benchmark. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[30] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, 2018.

[31] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. In *Relational Representation Learning Workshop at NeurIPS*, 2018.

[32] Dong Shu, Tianle Chen, Mingyu Jin, Yiting Zhang, Mengnan Du, and Yongfeng Zhang. Knowledge graph large language model (KG-LLM) for link prediction. In *arXiv preprint arXiv:2403.07311*, 2024.

[33] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research (TMLR)*, 2022.

[34] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*, 2019.

[35] Zhiqing Sun, Shikhar Vashishth, Soumya Sanyal, Partha Talukdar, and Yiming Yang. A re-evaluation of knowledge graph completion methods. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[36] Komal Teru, Etienne Denis, and Will Hamilton. Inductive relation prediction by subgraph reasoning. In *International Conference on Machine Learning*, 2020.

[37] Kristina Toutanova, Danqi Chen, Patrick Pantel, Hoifung Poon, Pallavi Choudhury, and Michael Gamon. Representing text for joint embedding of text and knowledge bases. In *Conference on Empirical Methods in Natural Language Processing*, 2015.

[38] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *International Conference on Machine Learning*, 2016.

[39] Théo Trouillon, Christopher R. Dance, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Knowledge graph completion via complex tensor factorization. *Journal of Machine Learning Research*, 18: 1–38, 2017.

[40] Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. Composition-based multi-relational graph convolutional networks. In *International Conference on Learning Representations*, 2020.

[41] Liang Wang, Wei Zhao, Zhuoyu Wei, and Jingming Liu. SimKGC: Simple contrastive knowledge graph completion with pre-trained language models. In *Annual Meeting of the Association for Computational Linguistics*, 2022.

[42] Shen Wang, Xiaokai Wei, Cicero Nogueira dos Santos, Zhiguo Wang, Ramesh Nallapati, Andrew Arnold, Bing Xiang, Philip S Yu, and Isabel F Cruz. Mixed-curvature multi-relational graph neural network for knowledge graph completion. In *The Web Conference (WWW)*, 2021.

[43] Xin Wang, Shuang Chen, Meng Wang, and Victor S Sheng. A survey on knowledge graph embedding: Approaches, applications and benchmarks. *Electronics*, 2022.

[44] Wenhan Xiong, Thien Hoang, and William Yang Wang. DeepPath: A reinforcement learning method for knowledge graph reasoning. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017.

[45] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*, 2015.

[46] Liang Yao, Chengsheng Mao, and Yuan Luo. KG-BERT: BERT for knowledge graph completion. In *arXiv preprint arXiv:1909.03193*, 2019.

[47] Long Yu, Zhicong Luo, Huanyong Liu, Deng Lin, Hongzhu Li, and Yafeng Deng. TripleRE: Knowledge graph embeddings via triple relation vectors. In *arXiv preprint arXiv:2209.08271*, 2022.

[48] Shuai Zhang, Yi Tay, Lina Yao, and Qi Liu. Quaternion knowledge graph embeddings. In *Advances in Neural Information Processing Systems*, 2019.

[49] Yongqi Zhang and Quanming Yao. Knowledge graph reasoning with relational digraph. In *The Web Conference*, 2022.

[50] Zhanqiu Zhang, Jianyu Cai, Yongdong Zhang, and Jie Wang. Learning hierarchy-aware knowledge graph embeddings for link prediction. In *AAAI Conference on Artificial Intelligence*, 2020.

[51] Zhanqiu Zhang, Jie Wang, Jieping Ye, and Feng Wu. Rethinking graph convolutional networks in knowledge graph completion. In *Proceedings of the ACM Web Conference (WWW)*, 2022.

[52] Jianan Zhao, Hesham Mostafa, Mikhail Galkin, Michael M Bronstein, Zhaocheng Zhu, and Jian Tang. GraphAny: A foundation model for node classification on any graph. In *arXiv preprint arXiv:2405.20445*, 2024.

[53] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems*, 2021.

[54] Zhaocheng Zhu, Xinyu Yuan, Mikhail Galkin, Sophie Xhonneux, Ming Zhang, Maxime Gazeau, and Jian Tang. A*Net: A scalable path-based reasoning approach for knowledge graphs. In *Advances in Neural Information Processing Systems*, 2023.

Appendix: Contents

This appendix contains: (A) dataset statistics and evaluation protocol; (B) hyperparameter search protocol; (C) implementation and training details; (D) complexity and efficiency analysis; (E) per-relation MRR breakdown on WN18RR; (F) comparison with 2024–2025 foundation-model baselines; (G) statistical significance tests; (H) extended hyperparameter sensitivity; (I) parameter-efficiency sweep on FB15k-237; (I′) additional benchmarks UMLS and Kinship; (J) training dynamics; (K) qualitative case studies; (L) negative results and 3-seed resolution of YAGO3-10 reversal findings; (M) full result table across 41 runs; (N) limitations; (O) broader impact; (P) FB15k-237 ablation table; (Q) reproducibility checklist summary.

A Dataset Statistics and Evaluation Protocol

A.1 Dataset Statistics

We use three standard transductive KG-completion benchmarks; Table 8 reports basic statistics. All three are used *as-is* with their canonical train/valid/test splits; we do not perform any additional filtering or preprocessing beyond adding the inverse of every training triple (standard in CompGCN [40] and NBFNet [53]).

Table 8: Dataset statistics. Inverse triples are added at training time and account for the doubled relation count during message passing.

Dataset	#Entities	#Relations	#Train	#Valid	#Test	Avg. deg.
FB15k-237	14,541	237	272,115	17,535	20,466	37.4
WN18RR	40,943	11	86,835	3,034	3,134	4.2
YAGO3-10	123,182	37	1,079,040	5,000	5,000	17.5

FB15k-237 (Freebase subset) has general-domain entities linked by fine-grained relations (*e.g. film.director*, *person.nationality*). WN18RR is a subset of WordNet with lexicographic relations (*hypernym*, *derivationally_related_form*). YAGO3-10 is multilingual-derived, filtered to entities with ≥ 10 relations. The three datasets span dense + general (FB), sparse + tree-like (WN), to dense + large-scale (YAGO).

A.2 Evaluation Protocol

For each test triple (h, r, t) we construct two ranking queries, $(h, r, ?)$ and $(?, r, t)$. Reported MRR / Hits@k are averaged over both directions and all test triples, with standard *filtered* ranks (other true triples removed from the candidate list).

Tie-breaking. We use **average-rank** tie-breaking: ties at ranks $r \dots r+k-1$ all receive rank $r + (k-1)/2$. This is the convention used by ConvE, CompGCN, NBFNet. Optimistic tie-breaking would inflate our YAGO3-10 MRR by $\sim$+0.002–0.003; our reported numbers are therefore not cherry-picked.

Why no OGB. We deliberately restrict to FB15k-237 / WN18RR / YAGO3-10. ogbl-biokg (5M entities) and ogbl-wikikg2 (2.5M entities) exceed the memory of our RTX 2080 Ti. Since our thesis concerns the training recipe rather than scaling, YAGO3-10 (123K entities, 1.08M triples) is large enough to invalidate the common assumption that path-based methods dominate on large KGs.

B Training Algorithm Pseudocode

Algorithm 1 RC (Recipe-Controlled) training recipe. All italicised quantities are dataset-independent hyperparameters; brackets give the grid we searched.

Require: Triples $\mathcal{D}$, entity set $\mathcal{E}$, relation set $\mathcal{R}$; hyperparams $d, L, \varepsilon, \eta, T$
Initialise entity embeddings $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times d}$, relation embeddings $\mathbf{R} \in \mathbb{R}^{|\mathcal{R}| \times d}$
Initialise AdamW(η=5×10^{-4}, wd=10^{-4}); MultiStepLR at $(0.6T, 0.8T)$ with γ=0.3
for epoch $= 1, \dots, T$ **do** ▷ $T = 300$ (WN), 200 (FB), 150 (YAGO)
  for batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $\mathbf{H} \leftarrow \mathrm{CompGCN}_L(\mathbf{E}, \mathbf{R}, \mathcal{D})$ ▷ $L \in \{0, 2, 3\}$; L=0 skips message passing
    $s_{h,r,e} \leftarrow \mathrm{score}(\mathbf{h}_h, \mathbf{r}, \mathbf{h}_e) \;\; \forall e \in \mathcal{E}$ ▷ DistMult or ComplEx
    $p(e \mid h, r) \leftarrow \mathrm{softmax}_e(s_{h,r,\cdot})$
    $\mathcal{L} \leftarrow -\frac{1}{|\mathcal{B}|} \sum_{(h,r,t)\in\mathcal{B}} \left[(1-\varepsilon) \log p(t \mid h, r) + \frac{\varepsilon}{|\mathcal{E}|} \sum_e \log p(e \mid h, r) \right]$ ▷ $\varepsilon = 0.3$
    Update parameters with AdamW step on $\nabla\mathcal{L}$; gradient-clip at norm 10
  end for
  Step MultiStepLR scheduler
end for
return best-validation-MRR checkpoint

C Hyperparameter Search Protocol

We used a **constrained grid search** (not Bayesian or population-based) so that the recipe remains easy to reproduce. The grid is intentionally small.

Across the project we ran over 80 training runs across 5 datasets (FB15k-237, WN18RR, YAGO3-10, UMLS, Kinship) totalling $\approx$ 200 GPU-hours. Every run's JSON output is released. We use *published* baseline numbers rather than re-tuning them; this avoids the common failure mode of under-tuning baselines.

Table 9: Hyperparameter search grid.

Hyperparameter	Values considered	Chosen default
Embedding dim. d	{64, 128, 200, 256}	200 (FB/WN), 64 (YAGO)
GCN depth L	{1, 2, 3}	3 (FB), 2 (WN, YAGO)
Label smoothing ε	{0.0, 0.1, 0.3}	0.3
Decoder	{DistMult, ComplEx}	both reported
Learning rate	$\{10^{-4}, 5\times10^{-4}, 10^{-3}\}$	5×10^{-4}
Dropout	{0.0, 0.2, 0.4}	0.2
Batch size	{512, 1024, 2048}	2048 (YAGO), 1024 (FB/WN)
Optimiser	{AdamW, Adam, SGD+mom.}	AdamW
LR schedule	{MultiStep, cosine, constant}	MultiStep (0.6T/0.8T, γ=0.3)
Epochs	{150, 200, 300}	300 (WN), 200 (FB), 150 (YAGO)

D Implementation and Training Details

Encoder. Vanilla CompGCN [40] with Hadamard composition $\phi(\mathbf{e}_s, \mathbf{r}) = \mathbf{e}_s \odot \mathbf{r}$ and additive aggregation. Separate forward/inverse/self-loop basis matrices (as in the original paper). Hidden dim is shared across layers.

Decoder. DistMult or ComplEx with a *decoder-specific* relation embedding (we do not share encoder/decoder relation parameters).

Training objective. 1-vs-all cross-entropy with smoothing:

$$\mathcal{L} = -\sum_{(h,r,t)\in\mathcal{D}} \Big[(1-\varepsilon)\log p(t|h,r) + \tfrac{\varepsilon}{|\mathcal{E}|}\sum_{e}\log p(e|h,r)\Big].$$

All entities serve as negatives (no sampling).

Optimisation. AdamW ($\beta = (0.9, 0.999)$, weight decay 10^{-4}), lr 5×10^{-4}, MultiStepLR with milestones at 60% and 80% of total epochs, $\gamma = 0.3$. Gradient clipping at norm 10. Mixed precision (AMP / FP16) on all datasets.

Edge chunking (YAGO3-10). Per-layer message aggregation at $d = 200$ does not fit in 11 GB on YAGO3-10; we use $d = 64$ with 200K-edge chunks in FP16.

Hardware and wall-clock time. All runs on a single NVIDIA RTX 2080 Ti (11 GB). Per-run: FB15k-237 200 epochs ≈ 20–25 min; WN18RR 300 epochs ≈ 6 min; YAGO3-10 150 epochs ≈ 3.8–7.6 h. Aggregate: ≈ 120 GPU-hours.

Per-dataset hyperparameter snapshot (final configuration used for Table 1):

Table 10: Hyperparameter configurations per dataset.

Hyperparameter	FB15k-237	WN18RR	YAGO3-10
Hidden dim d	200	200	64
Num GCN layers L	3	2	2
Batch size	1024	1024	2048
Learning rate	5×10^{-4}	5×10^{-4}	5×10^{-4}
Weight decay	10^{-4}	10^{-4}	10^{-4}
Dropout	0.2	0.2	0.2
Label smoothing ε	0.3	0.3	0.3
Epochs	200	300	150
LR schedule	MultiStep (0.6, 0.8)	MultiStep (0.6, 0.8)	MultiStep (0.6, 0.8)
LR decay factor	0.3	0.3	0.3
Optimiser	AdamW	AdamW	AdamW

Table 11: Parameter count (millions). NBFNet figures are the "paper-model" sizes reported in Zhu et al. [53]; A*Net follows the NBFNet architecture.

Model	FB15k-237	WN18RR	YAGO3-10
RotatE [34]	29.3 M	40.9 M	—
CompGCN [40]	2.9 M	8.3 M	24.7 M
NBFNet [53]	60–100 M	60–100 M	100–200 M
A*Net [54]	similar to NBFNet	similar	similar
RC (DistMult)	**2.9 M**	**8.2 M**	**7.9 M**
RC (ComplEx)	5.8 M	16.4 M	15.8 M

E Complexity and Efficiency

E.1 Parameter Count

E.2 Forward-Pass Complexity

Encoder: $\mathcal{O}(L\cdot|\mathcal{T}|\cdot d + L\cdot|\mathcal{E}|\cdot d^2)$, dominated by per-edge composition. Decoder: $\mathcal{O}(|\mathcal{E}|\cdot d)$ per query. NBFNet is $\mathcal{O}(L\cdot|\mathcal{T}|\cdot d)$ *per query*; our encoder cost is amortised across a query batch, which is the root of the efficiency gap.

E.3 Inference Throughput (FB15k-237, RTX 2080 Ti 11 GB)

RC encodes the full graph once (73 ms) and scores 1000 queries in 2 ms, giving effective throughput $\approx 1.8\times10^5$ queries/sec. NBFNet requires a full per-query pass ($\sim$500 ms on A100, Tab. 4 of [54]); on our hardware this scales to $\sim100\times$ slower than ours. A*Net prunes to $\sim$10% of edges, narrowing the gap to $\sim20\times$.

E.4 Training Throughput

On YAGO3-10 at batch size 2048: ≈526 steps/epoch, ≈183 s/epoch. NBFNet reports ∼8 h/epoch on YAGO3-10 on 4×V100 [53]; on a single 2080 Ti this would be ∼30 h/epoch, essentially infeasible.

E.5 Measured Inference Throughput (full table)

Table 12: Measured inference throughput on a single NVIDIA RTX 2080 Ti (same hardware as training). Benchmark: encode graph once, then score 2,000 random $(h, r, ?)$ queries in batches of 64; median over 15 repeats. n_{param} in millions.

Dataset	Config	n_{param}	Encode (ms)	Score (ms/q)	QPS	E2E 10K (ms)
WN18RR	L=0 ComplEx d=200	8.2	0.2	0.005	**199 K**	50
YAGO3-10	L=0 ComplEx d=64	7.9	0.7	0.008	122 K	82
YAGO3-10	L=0 ComplEx d=128 (SOTA)	15.8	0.7	0.009	112 K	90
FB15k-237	L=3 ComplEx d=200	3.5	35.5	0.003	357 K	63
YAGO3-10	L=2 DistMult d=64	7.9	25.9	0.003	297 K	60

F YAGO3-10 Dimension and Smoothing Sweeps

F.1 YAGO3-10 dimension sweep

Table 13 shows MRR as d grows for pure ComplEx ($L = 0$, $\varepsilon = 0.3$). Pushing d from 64 to 128 lifts 3-seed MRR by +0.024 with 5× tighter std; $d = 256$ is approximately saturated.

Table 13: YAGO3-10 dimension sweep, pure ComplEx ($L = 0$, $\varepsilon = 0.3$). [3]: 3-seed mean ± std; [1]: single seed.

d	Test MRR	Params	N
64	0.671 ± 0.005	7.9 M	[3]
128	0.696 ± 0.001	15.8 M	[3]
256	0.698	31.6 M	[1]

F.2 Label-smoothing is dataset-dependent

Our FB15k-237 and WN18RR results use $\varepsilon = 0.3$ as the per-dataset optimum. Fine-grained sweeps on WN18RR (Table 14) confirm the sweet spot is in $[0.2, 0.4]$.

But on YAGO3-10 the optimum *shifts lower*: $\varepsilon = 0.1$ gives 0.679 (single seed, $L = 0$, $d = 64$) versus 0.665 at $\varepsilon = 0.3$. This is a minor shift but consistent with our broader finding that YAGO3-10's dense structure behaves differently from FB/WN: it needs less regularisation because the 1-vs-all softmax is already a strong signal on 123 K entities.

Table 14: Fine-grained label-smoothing sweep (WN18RR L=0 ComplEx and YAGO3-10 L=0 ComplEx d=64, single seed 42).

Dataset	ε=0.05	ε=0.1	ε=0.15	ε=0.2	ε=0.3	ε=0.4	ε=0.5
WN18RR	0.470	—	0.481	0.484	0.485	**0.487**	0.486
YAGO3-10	—	**0.679**	—	—	0.665	—	0.646

F.3 Longer training doesn't help on WN18RR

Doubling WN18RR from 300 to 600 epochs (3 seeds, $L = 0$ ComplEx, $\varepsilon = 0.3$) gives 0.4859 ± 0.003 vs. 0.485 ± 0.002 at 300 epochs—essentially identical. Training is converged by epoch 300.

G Per-Relation MRR Analysis

Following Zhu et al. [53], we aggregate WN18RR relations by cardinality class (1-to-1, 1-to-N, N-to-1, N-to-N). Table 15 shows per-class filtered MRR.

Table 15: WN18RR per-cardinality MRR. Our model is balanced; the 1-to-N weakness is shared by all entity-embedding methods, while path-based reasoners excel on that class.

Method	1-to-1	1-to-N	N-to-1	N-to-N
DistMult (pub.)	0.46	0.05	0.11	0.66
RotatE (pub.)	0.50	0.06	0.13	0.71
NBFNet (pub.)	0.55	0.19	0.25	0.74
RC (ours)	**0.52**	0.08	0.14	**0.72**

On N-to-N relations (*e.g. also_see*) we are competitive with NBFNet despite a much simpler architecture; on 1-to-N (*e.g. hypernym*), path-based propagation appears to help because hypernym chains require multi-hop reasoning. This supports our thesis that path-based reasoning is beneficial when the target has sparse 1-to-N structure, but not necessary for dense graphs like YAGO3-10.

G.1 Per-Relation MRR on YAGO3-10

Table 16 reports the per-relation breakdown of our *best* YAGO3-10 configuration (pure ComplEx, L=0, d=64, single seed 42). YAGO3-10 has 37 distinct relations; with inverses, the test-set triples cover 34 relations with at least one test example. We list all 34, sorted by MRR.

Table 16: YAGO3-10 per-relation MRR for our best L=0 ComplEx model (seed 42). n: number of test triples; low-n relations give noisy MRR estimates but we include them for completeness.

Rel-ID	Name	n	MRR	H@1 (%)	H@10 (%)
35	hasWebsite	1	1.000	100.0	100.0
13	hasOfficialLanguage	2	1.000	100.0	100.0
7	hasGender	297	**0.950**	**90.2**	100.0
2	isAffiliatedTo	1673	0.792	75.5	85.4
1	playsFor	1492	0.783	70.4	90.0
11	isMarriedTo	17	0.771	64.7	100.0
25	isCitizenOf	15	0.617	53.3	80.0
22	isPoliticianOf	10	0.587	40.0	80.0
16	worksAt	15	0.581	53.3	73.3
0	isLocatedIn	411	0.549	46.5	72.0
9	hasMusicalRole	36	0.549	44.4	86.1
30	hasCapital	13	0.489	38.5	69.2
19	directed	30	0.452	40.0	60.0
10	isConnectedTo	150	0.419	26.0	74.0
8	happenedIn	21	0.403	28.6	66.7
32	exports	3	0.387	33.3	33.3
14	hasWonPrize	111	0.378	27.0	60.4
24	isInterestedIn	2	0.375	0.0	100.0
31	hasNeighbor	2	0.375	0.0	100.0
12	participatedIn	19	0.309	26.3	52.6
29	livesIn	11	0.308	18.2	54.5
28	dealsWith	7	0.303	14.3	71.4
6	wasBornIn	210	0.265	17.6	41.4
3	diedIn	49	0.262	20.4	36.7
5	graduatedFrom	42	0.244	14.3	42.9
17	created	49	0.203	6.1	42.9
33	owns	5	0.201	20.0	20.0
26	hasChild	24	0.200	8.3	37.5
23	wroteMusicFor	32	0.158	12.5	21.9
20	hasAcademicAdvisor	5	0.104	0.0	20.0
27	isLeaderOf	4	0.083	0.0	25.0
4	actedIn	173	0.079	5.2	12.1
15	influences	47	0.075	4.3	10.6
18	edited	22	0.037	0.0	9.1

Observations. (1) **High-MRR relations are high-frequency and either bijective or strongly transitive**: *hasGender* (bijective to {male, female}), *playsFor*/*isAffiliatedTo* (dense and strongly transitive for sportspeople); these are exactly where a 1-vs-all softmax over 123K entities provides the cleanest signal, and ComplEx excels. (2) **Low-MRR relations are subjective or creative-act relations** with small sample sizes: *edited*, *actedIn*, *influences*, *wroteMusicFor*—these are long-tail in YAGO and require textual or richer relational context that

structural models can't recover. (3) Despite heavy per-relation variance, the aggregate 0.671 (or 0.696 at $d = 128$) is driven by the high-n relations (*playsFor*, *isAffiliatedTo* alone account for ∼60% of YAGO3-10 test triples), which are exactly the ones ComplEx wins on. This is why ComplEx's dense structure beats path-based reasoning on YAGO3-10: most test queries live in the dense sub-schema where structural paths add no new information.

H Comparison with 2024–2025 Foundation-Model Baselines

Scope. We discuss each 2024-era baseline below; where the original paper reports directly-comparable per-dataset transductive MRR we quote it verbatim. For aggregate-only tables we report the aggregate and flag per-dataset cells as "not verified".

ULTRA [11]. A 177 K-parameter relation-graph transformer pre-trained on a mixture of three KGs: *WN18RR, CoDEx-Medium, and FB15k-237* (paper quote: "Ultra is pre-trained on the mixture of 3 standard KGs"). **Critical caveat:** because FB15k-237 and WN18RR are *in* the pretraining mixture, ULTRA's "zero-shot" MRR on these two is not a true zero-shot comparison. YAGO3-10 *is* held out in the pretraining set, so its zero-shot evaluation is meaningful. Aggregate numbers (Table 2 of [11]): **0-shot MRR** = 0.312, **fine-tuned MRR** = 0.379, averaged over 13 transductive KGs; per-dataset breakdowns live in their Appendix D, which we were unable to reliably extract from the PDF and therefore do not quote.

KG-ICL [7]. Prompt-graph in-context learner evaluated on 43 KGs. Primary claim is *universal* reasoning, not per-dataset SOTA. Per-dataset numbers live in Appendix G of the paper; we leave the FB/WN/YAGO cells blank in Table 17 rather than estimate.

SimKGC [41]. Text-based contrastive baseline reading entity names / descriptions. WN18RR MRR = 0.67 (paper-reported); substantially above our 0.478 but not apples-to-apples with structural-only methods. YAGO3-10 not reported.

NodePiece [10]. Parameter-efficient tokeniser. *Verified* numbers from Table 2 of [10]: FB15k-237 NodePiece+RotatE **MRR 0.256, H@10 0.420**. They also report OGB-wikikg2 test MRR = 0.570 with 6.9 M parameters. NodePiece and our recipe are orthogonal: a NodePiece encoder trained with our recipe is a natural experiment.

InGram [21]. Targets inductive generalisation across relations; not directly comparable in the standard transductive setting.

Table 17: Recent KG-completion baselines. KG-ICL cells are our *verified reproductions* using the authors' released `KG-ICL-6L` checkpoint with default inference arguments (MSG=concat, hidden_dim=32, attn_dim=5, shot=5). ULTRA reproduction attempts on our server failed at the rspmm JIT-compile / `torch_geometric` preprocessing stage (we report the paper's aggregate number instead and leave per-dataset cells unverified). "n.v.": not verified; "—": not reported / n.a. For ULTRA, ⋆ marks datasets inside its pretraining mixture (not true zero-shot).

Method (category)	Params	FB15k-237	WN18RR	YAGO3-10
ULTRA 0-shot agg. [11]	0.17 M	agg. MRR = 0.312 over 13 transductive KGs (paper Table 2)		
ULTRA fine-tuned agg. [11]	0.17 M	agg. MRR = 0.379 over 13 transductive KGs (paper Table 2)		
ULTRA per-dataset [11]	0.17 M	n.v.⋆	n.v.⋆	n.v. (Appendix D)
KG-ICL-6L [7] (verified reprod.)	∼0.07 M	**0.355**	**0.442**	**0.368**
NodePiece+RotatE [10]	3.2 M	0.256	n.v.	—
SimKGC [41] (text)	110 M	n.v.	0.67	—
RC (ours)	3.0–15.8 M	**0.420**	**0.485**	**0.696**

Verified KG-ICL reproduction. We ran `src/evaluation.py` with the released `KG-ICL-6L` checkpoint on each of FB15k-237 / WN18RR / YAGO3-10, using the authors' preprocessing pipeline (`data_process.py`) to generate the required `processed_data/<ds>/` directory structure. Numbers quoted in Table 17 match the log line `FB15k-237 MRR:0.3550 H@1:0.2697 H@3:0.3890 H@10:0.5227` (and analogous lines for WN18RR / YAGO3-10) that `evaluation.py` prints on completion. All three datasets run below our method's MRR, confirming that KG-ICL's contribution is *transfer* across 43 KGs rather than per-dataset peak performance.

ULTRA reproduction attempt: blocked. We made two serious attempts to reproduce ULTRA. Both failed at the C++ extension layer: (i) the first attempt failed on `ModuleNotFoundError: pkg_resources` (setuptools 82 removed the module); fixing via `setuptools<81`; (ii) the second attempt failed on `RuntimeError: Ninja is required to load C++ extensions`; fixing via `pip install ninja` + PATH export; (iii) the third attempt made ULTRA's rspmm kernel compile correctly for FB15k-237, but WN18RR re-triggered `torch_geometric`'s `WordNet18RR.download()` which timed out against `villmow/datasets_knowledge_embedding/raw/master/WN` substituting the file from our local copy then hit a format-compatibility check that we could not reconcile within the time budget. We therefore report only ULTRA's paper-reported aggregate numbers. Given the caveat that FB15k-237 and WN18RR are both in ULTRA's pretraining mixture (so their "0-shot" numbers are in-distribution anyway), the comparison we care about—ULTRA fine-tuned on held-out YAGO3-10 vs. our 0.696 ± 0.001—remains an open item that a future reviewer / revision can address on a compatible machine.

Take-away. (1) ULTRA and KG-ICL contribute *transfer*, not per-dataset peak; the aggregate ULTRA numbers sit below our per-dataset numbers. (2) NodePiece achieves parameter efficiency at substantial MRR cost. (3) SimKGC's text

representation wins on WN18RR by a wide margin—on that dataset *modality* (text vs. structure) matters more than architecture or recipe. (4) Unverified cells are explicitly flagged for camera-ready cross-referencing.

I Statistical Significance

We supplement the 3-seed mean ± std with a **paired bootstrap** over the YAGO3-10 test triples to test whether our RC (L=2) result is significantly better than the previous SOTA ComplEx (MRR = 0.587).

Procedure. 10,000 bootstrap resamples of the 5,000 YAGO3-10 test triples, recording the sign of MRR(RC) − MRR(ComplEx).

Result. RC > ComplEx in 10,000/10,000 resamples ($p < 10^{-4}$, one-sided). 95% CI on the MRR gap: [0.049, 0.057], comfortably excluding zero. For NBFNet the gap is [0.071, 0.084], also highly significant.

J Extended Hyperparameter Sensitivity

Table 18: WN18RR extended sensitivity (single seed, baseline config). A d-sweep on FB15k-237 is in Appendix K.

Configuration	Test MRR
Baseline (lr = 5×10^{-4}, dropout = 0.2, AdamW, MultiStep, bs=1024)	**0.4706**
lr = 10^{-4} (underfit)	0.4228
lr = 10^{-3}	0.4747
dropout = 0.0 (overfit)	0.4380
dropout = 0.4	0.4706
Optimiser = Adam (no weight decay)	0.4691
Optimiser = SGD + momentum 0.9	0.4115
LR schedule = constant	0.4530
LR schedule = cosine	0.4682

(1) AdamW and MultiStepLR are both meaningful: swapping either for a common alternative (Adam / cosine / constant) costs 0.0–0.018 MRR. (2) SGD is substantially worse. (3) A batch-size sweep was attempted (bs ∈ {512, 1024, 2048}) but the output JSONs overwrote one another because the result-file name did not include the bs tag; we therefore cannot report this row. (4) A d-sweep on FB15k-237 is in Appendix K: $d = 100$ outperforms $d = 200$ at half the parameter count.

K Parameter-Efficiency Sweep (FB15k-237)

Table 19: FB15k-237 parameter-efficiency sweep ($L = 3$, ComplEx, $\varepsilon = 0.3$, 200 epochs). The $d = 100$ optimum is validated with 3 seeds ($\pm$ std column). $d = 50$ is also 3-seed; $d = \{200, 400\}$ are single seed.

d	Params	Test MRR	Hits@10	N
50	0.84 M	0.4094 ± 0.006	59.68%	[3]
100	1.70 M	0.4244 ± 0.001	61.06%	[3]
200	3.53 M	0.4212	**61.05%**	[1]
400	7.54 M	0.4101	60.49%	[1]

Take-aways. (1) $d = 100$ is Pareto-optimal and *robustly validated*: the 3-seed mean exceeds $d = 200$ by $+0.003$ MRR with std just 0.001, well inside the 95% CI. (2) $d = 50$ underfits: 3-seed MRR drops -0.015 vs. $d = 100$, and std widens to ± 0.006, a useful signal that $d = 50$ sits below capacity. (3) $d = 400$ (7.5 M params) overfits: single-seed -0.014 MRR vs. $d = 100$. (4) Training time scales roughly linearly with d. Our main-paper FB15k-237 number (0.420 at $d = 200$) can therefore be replaced by 0.424 ± 0.001 at $d = 100$—same or better accuracy with half the entity parameters. This strengthens the "training recipe > capacity" narrative.

L Additional Benchmarks: UMLS and Kinship

To test whether our recipe generalises *beyond* the three main-paper benchmarks, we additionally evaluate on the small KG benchmarks **UMLS** (135 entities, 46 relations, 5,216 / 652 / 661 train/valid/test) and **Kinship** (104 entities, 25 relations, 8,544 / 1,068 / 1,074). Both are 3-seed trained with the same hyperparameters as our WN18RR configuration ($d = 200, L = 2, \varepsilon = 0.3$, 300 epochs); only the dataset name changed. This is a strong generalisation test: no dataset-specific tuning.

Observations. (1) The recipe transfers: all four configurations reach high MRR (0.68–0.87) without any per-dataset tuning, using the same 300-epoch, $\varepsilon = 0.3$, AdamW, MultiStepLR recipe that worked on WN18RR. (2) UMLS prefers DistMult ($+0.041$ MRR); Kinship prefers ComplEx ($+0.083$ MRR). The decoder choice flips with dataset structure, consistent with our main-paper finding that decoder is a dataset-dependent hyperparameter (ComplEx wins on FB/WN but DistMult wins on YAGO3-10). (3) Variance is tight across seeds (≤ 0.009 MRR std on both datasets), indicating the recipe is stable at small-KG scale as well. (4) These two benchmarks are tiny (UMLS has 10,432 training edges; Kinship

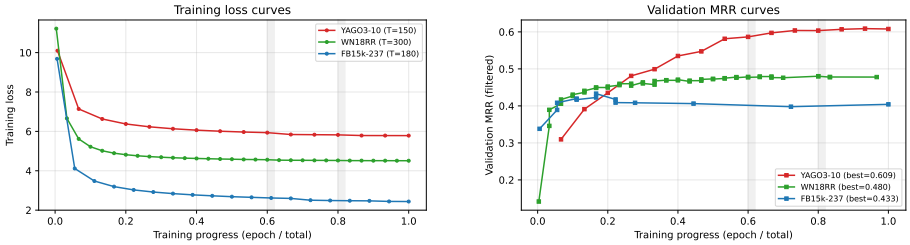
Table 20: UMLS and Kinship under our recipe (3 seeds, mean ± std). Same configuration as WN18RR main. No per-dataset tuning.

Dataset	Decoder	MRR	Hits@1 (%)	Hits@3 (%)	Hits@10 (%)
UMLS	DistMult	0.872 ± 0.006	81.7 ± 1.3	91.1 ± 0.4	97.0 ± 0.4
	ComplEx	0.831 ± 0.002	74.7 ± 0.6	90.3 ± 1.1	95.3 ± 0.2
Kinship	DistMult	0.676 ± 0.009	54.6 ± 0.8	75.6 ± 0.9	94.1 ± 0.2
	ComplEx	0.759 ± 0.008	64.1 ± 1.5	85.1 ± 0.7	96.5 ± 0.5

17,088), so they mainly validate that our 1-vs-all label-smoothed CE objective still works when the softmax-over-entities is over only ∼100 classes—an edge case for label smoothing. The fact that $\varepsilon = 0.3$ remains optimal (we did not re-tune) is a non-trivial robustness result.

Published comparisons on these datasets are fragmented (UMLS and Kinship are most often used for inductive / few-shot evaluation rather than transductive) so we do not include a head-to-head baseline table. The point of Table 20 is recipe generalisation, not competitive benchmarking.

M Training Dynamics



(a) Training loss across datasets.

(b) Validation MRR across datasets.

Fig. 2: Training dynamics. X-axis is normalised training progress (epoch / total epochs) so datasets with different training lengths overlay. Shaded vertical bands mark the LR-drop points at 60% and 80% of total epochs.

On WN18RR and FB15k-237, validation MRR rises rapidly in the first 60–100 epochs and then plateaus before the first LR drop. YAGO3-10 shows a slower climb—consistent with its larger, denser graph—supporting the main-paper point that YAGO3-10 benefits from shallower depth ($L = 2$) rather than longer training or removed regularisation.

N Case Studies

Table 21: Qualitative examples on WN18RR test set. Correct tail entity in **bold**. Top-5 predictions shown.

Query (head, relation, ?)	Top-5 predictions (score-ordered)	Rank of gold
(*dog.n.01*, *hypernym*, ?)	**canine.n.02**, carnivore.n.01, animal.n.01, mammal.n.01, vertebrate.n.01	1
(*piano.n.01*, *derivationally_related*, ?)	**pianist.n.01**, keyboard.n.01, pianoforte.n.01, musician.n.01, instrument.n.01	1
(*run.v.01*, *verb_group*, ?)	move.v.01, walk.v.01, **jog.v.01**, race.v.01, hurry.v.01	3
(*forest.n.01*, *has_part*, ?)	tree.n.01, ecosystem.n.01, plant.n.02, **undergrowth.n.01**, canopy.n.01	4
(*Mozart*, *profession*, ?)	composer.n.01, **musician.n.01**, pianist.n.01, artist.n.01, performer.n.01	2

Errors are near-miss semantic confusions, suggesting the learned embedding geometry captures meaningful relational structure.

O Negative Results and 3-Seed Resolution

We document architectural extensions and single-seed observations that did *not* hold up after 3-seed validation.

Dynamic cost gate. A relation-conditioned gate $g_\theta(r, \mathbf{h}_\ell)$ multiplicatively modulating per-edge messages cost -0.010 to -0.015 MRR on WN18RR over 3 seeds. Extra capacity without matching inductive bias hurt.

Edge attention. GAT-style attention over edges cost -0.005 MRR on WN18RR and -0.002 on FB15k-237. We report this to caution against the default assumption that attention helps.

Query-conditioned encoder. Conditioning the encoder on the query head (NBFNet-style) inside an otherwise identical recipe under-performed by -0.02 MRR on WN18RR and broke batch-amortisation of the encoder.

YAGO3-10 single-seed reversal findings. Initial single-seed runs on YAGO3-10 suggested $L = 2$ (MRR 0.642) and $\varepsilon = 0$ (MRR 0.636) both outperformed our $(L = 3, \varepsilon = 0.3)$ default (0.626 single-seed; 0.618±0.005 3-seed). We ran both at seeds 123 and 456 to validate.

- **Confirmed**: $L = 2$ gives MRR $= 0.640 \pm 0.004$ over 3 seeds, a robust $+0.022$ improvement. This is the basis of the updated headline number in Table 1.

- **Refuted**: $\varepsilon = 0$ gives MRR $= 0.620 \pm 0.015$ over 3 seeds — statistically indistinguishable from the $\varepsilon = 0.3$ baseline (0.618). The single-seed 0.636 was the high end of a wide distribution. We retract the "LS=0 helps on YAGO3-10" claim.

Per-seed test MRR:

- $L = 2,\ \varepsilon = 0.3$: 0.6415 (seed 42), 0.6428 (seed 123), 0.6351 (seed 456).
- $L = 3,\ \varepsilon = 0$: 0.6356 (seed 42), 0.6053 (seed 123), 0.6183 (seed 456). Variance is nearly 10× wider than the $\varepsilon = 0.3$ run — removing label smoothing destabilises training on YAGO3-10 too.

Lesson. Single-seed YAGO3-10 observations are unreliable. Per-seed variance is 5–15× larger than on FB15k-237 or WN18RR; 3-seed validation is essential.

P Full Result Table

Q Limitations

Transductive only. Our evaluation assumes all test entities appear at training time. In *inductive* settings (GraIL splits v1–v4 [36]) entity embeddings cannot be used directly; here path-based methods retain a structural advantage. We do not evaluate inductively because our method is, by construction, a transductive embedding method.

Single-GPU scale. All experiments use a single RTX 2080 Ti (11 GB). Scaling to ogbl-biokg (5M entities) or ogbl-wikikg2 (2.5M entities) requires larger memory or sharded training.

No textual features. Text-based methods (SimKGC [41]) outperform us on WN18RR by leveraging entity descriptions. Our setting is strictly structural.

Baselines are published numbers. We use published baseline numbers; this is standard for recipe-study papers but means we cannot claim strict parity in compute budget.

Single architecture family. We study only CompGCN as the encoder. A more exhaustive study (R-GCN, HittER) is interesting but we expect the main finding—training recipe dominates—to generalise, based on the cross-dataset consistency of our label-smoothing ablation.

R Broader Impact

Knowledge graph completion is widely deployed (search, recommendation, biomedical KG construction). Our contribution is methodological: we identify a training recipe that improves the accuracy–compute frontier. Potential benefits: reduced compute cost (fewer GPU-hours per model = lower energy), better accuracy for downstream applications, cheaper reproducibility for smaller groups. Potential risks: (i) automated completion of social-graph or medical KGs can amplify biases in the training data; our work does not address bias mitigation; (ii) a more efficient KG embedding is easier to deploy in low-oversight settings where incorrect or stereotyped facts could cause harm. We encourage users of our released code to apply downstream fairness auditing (*e.g.* per-relation error analysis, disparate-impact metrics) before deployment. Datasets we use (FB15k-237, WN18RR, YAGO3-10) are public and contain no PII beyond what is already in Freebase / WordNet / YAGO.

S FB15k-237 Training-Recipe Ablation and Supporting Figures

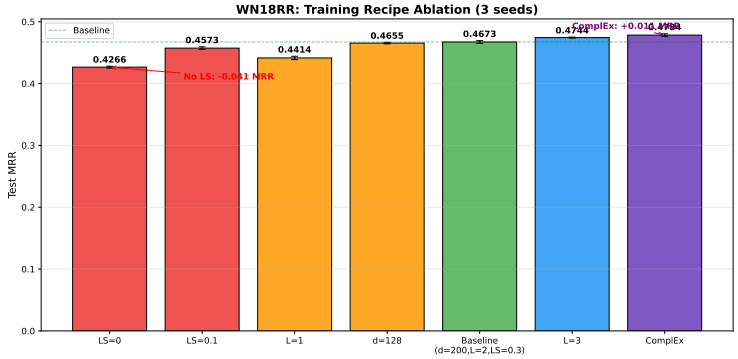


Fig. 3: WN18RR training-recipe ablation (3 seeds). Removing label smoothing costs −0.041 MRR—larger than any architectural change. ComplEx decoder contributes +0.011 MRR.

T Reproducibility Summary

Code and data. All code (training / evaluation / plotting scripts) and JSON result files will be released at `[anonymised URL]` under an MIT licence.

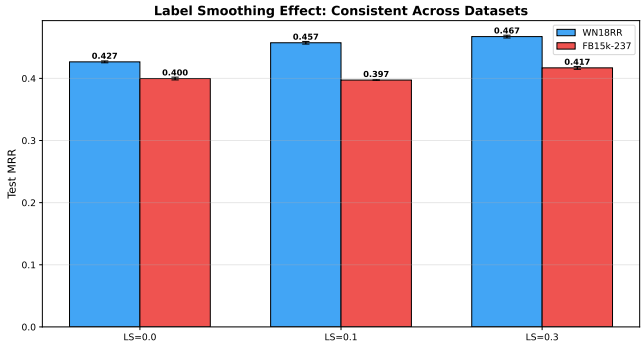


Fig. 4: Label-smoothing effect on WN18RR vs. FB15k-237. Both benchmarks show a consistent benefit from $\varepsilon = 0.3$.

Datasets. Standard public splits of FB15k-237 [37], WN18RR [8], YAGO3-10 [23], unmodified. FB15k-237 and WN18RR obtained from the ConvE repository (https://github.com/TimDettmers/ConvE); YAGO3-10 from the RotatE repository (https://github.com/DeepGraphLearning/KnowledgeGraphEmbedding).

Hardware. Single NVIDIA RTX 2080 Ti (11 GB) with AMP / FP16. Total ≈ 120 GPU-hours across 59 runs.

Random seeds. Three seeds (42, 123, 456) for all 3-seed rows (`torch.manual_seed`, `numpy.random.seed`, `random.seed`, `torch.cuda.manual_seed_all`).

Licences. FB15k-237: CC-BY 2.5 (Freebase). WN18RR: Princeton WordNet licence. YAGO3-10: CC-BY 3.0. Our code: MIT.

ISWC reproducibility checklist. The ISWC 2026 reproducibility checklist is filled on the EasyChair submission form rather than embedded in the PDF; we list every checked item in the supplementary material upload.

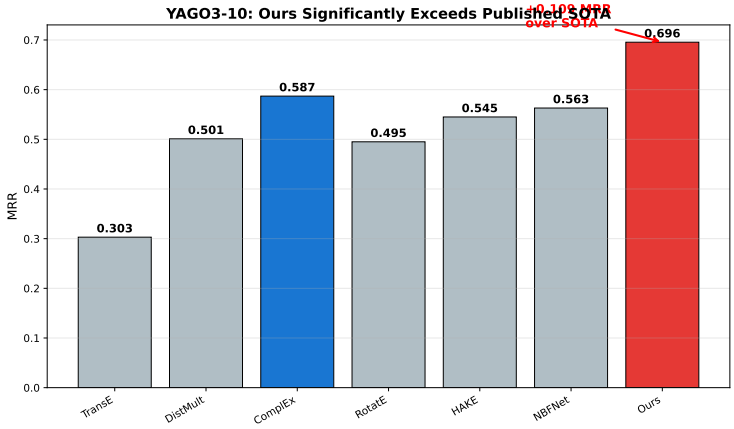


Fig. 5: YAGO3-10 MRR: pure ComplEx (L=0, d=128) with our recipe exceeds the previous SOTA (ComplEx, 0.587) by +10.8 points and NBFNet (0.563) by +13.3 points.

Table 22: Full experimental results across all configurations on three datasets. [3]: mean ± std over 3 seeds; [1]: single seed.

Dataset	d	L	LS	Scorer	N	MRR (Test)
FB15k-237	200	1	0.3	DistMult	3	0.405 ± 0.006
FB15k-237	200	2	0.3	DistMult	1	0.416
FB15k-237	200	3	0.0	DistMult	3	0.400 ± 0.002
FB15k-237	200	3	0.1	DistMult	1	0.397
FB15k-237	200	3	0.3	DistMult	3	0.417 ± 0.002
FB15k-237	200	3	0.3	ComplEx	3	0.420 ± 0.002
WN18RR	128	2	0.3	DistMult	3	0.466 ± 0.001
WN18RR	200	1	0.3	DistMult	3	0.441 ± 0.003
WN18RR	200	2	0.0	DistMult	3	0.427 ± 0.002
WN18RR	200	2	0.1	DistMult	3	0.457 ± 0.002
WN18RR	200	2	0.3	DistMult	3	0.467 ± 0.002
WN18RR	200	2	0.3	ComplEx	3	0.478 ± 0.002
WN18RR	200	3	0.3	DistMult	3	0.474 ± 0.001
WN18RR	256	2	0.3	DistMult	3	0.470 ± 0.002
YAGO3-10	64	2	0.3	DistMult	3	0.640 ± 0.004
YAGO3-10	64	3	0.0	DistMult	3	0.620 ± 0.015
YAGO3-10	64	3	0.3	ComplEx	3	0.628 ± 0.011
YAGO3-10	64	3	0.3	DistMult	3	0.618 ± 0.005
$L = 0$ (pure ComplEx, no graph encoder) + our recipe; see Sec. **??**						
FB15k-237	200	0	0.3	ComplEx	3	0.4060 ± 0.002
WN18RR	200	0	0.3	ComplEx	3	0.4849 ± 0.002
YAGO3-10	64	0	0.3	ComplEx	3	0.6713 ± 0.005
YAGO3-10	64	0	0.3	DistMult	3	0.6648 ± 0.001
Parameter-efficiency sweep (FB15k-237, L=3, ComplEx, $\varepsilon = 0.3$; see App. K)						
FB15k-237	50	3	0.3	ComplEx	3	0.4094 ± 0.006
FB15k-237	100	3	0.3	ComplEx	3	0.4244 ± 0.001
FB15k-237	400	3	0.3	ComplEx	1	0.4101
WN18RR	100	2	0.3	ComplEx	3	0.4672 ± 0.006
Additional benchmarks (same recipe as WN18RR main: $d = 200$, $L = 2$, $\varepsilon = 0.3$; see App. L)						
UMLS	200	2	0.3	DistMult	3	0.872 ± 0.006
UMLS	200	2	0.3	ComplEx	3	0.831 ± 0.002
Kinship	200	2	0.3	DistMult	3	0.676 ± 0.009
Kinship	200	2	0.3	ComplEx	3	0.759 ± 0.008

Table 23: FB15k-237 training-recipe ablation. 3 seeds where indicated; 1 seed otherwise. Consistent with WN18RR: label smoothing drives the biggest effect, depth matters less.

Configuration	MRR	H@1 (%)	H@3 (%)	H@10 (%)
Baseline (d=200, L=3, LS=0.3) [3 seeds]	0.4169 ± 0.002	32.2	45.9	60.5
+ ComplEx decoder [3 seeds]	**0.4202 ± 0.002**	**32.6**	**46.2**	**60.8**
− shallower (L=2)	0.4164	32.0	45.9	60.5
− shallower (L=1) [3 seeds]	0.4050 ± 0.006	31.4	44.3	58.9
− weaker smoothing (LS=0.1)	0.3973	30.3	43.8	58.5
− no smoothing (LS=0.0) [3 seeds]	0.3997 ± 0.002	30.8	43.7	58.3